

MuJoCo 使用指南

(Windows 平台・版本 3.14.0)



魏舟

目录

第一章	MuJoCo 简介与 Windows 环境准备	1
1.1	引擎定位与设计特点	1
1.2	三种使用方式	2
1.3	Windows 环境要求	3
1.4	软件清单与安装顺序	4
1.5	目录规划与环境自检	5
第二章	Python 环境与 mujoco 库安装	6
2.1	Python 版本选择与安装	6
2.1.1	版本要求	6
2.1.2	三种安装方式	6
2.1.3	验证 Python 与多版本共存	7
2.2	虚拟环境	7
2.2.1	为什么必须用	7
2.2.2	conda: 跨语言环境	7
2.3	安装 mujoco	7
2.3.1	基本安装	7
2.3.2	国内镜像加速	8
2.3.3	不要装 mujoco-py	8
2.4	安装验证	8
2.4.1	四步自检	8
2.4.2	跑通第一个模型	9
2.4.3	打开交互窗口	9
2.5	与周边生态的关系	10
2.6	常见安装问题	11
第三章	Python 基础用法与可视化	12
3.1	模型与数据	12
3.1.1	两个核心对象	12

3.1.2	加载模型	12
3.1.3	常用字段	13
3.2	推进仿真	14
3.2.1	三个入口	14
3.2.2	状态存取与关键帧	14
3.2.3	一个完整的受控例子	14
3.3	命名访问	15
3.3.1	按名字拿数组	15
3.4	交互式可视化	16
3.4.1	三种用法	16
3.4.2	passive 查看器	16
3.4.3	往画面里加自定义图形	18
3.5	离屏渲染与录像	18
3.5.1	基本用法	19
3.5.2	Windows 上的后端限制	19
3.6	模型转换与保存	20
3.7	常见运行期问题	21
第四章	C++ 开发环境与第一个程序	22
4.1	获取预编译库	22
4.1.1	下载	22
4.1.2	先跑通官方程序	22
4.2	编译工具链	23
4.2.1	Visual Studio 2022	23
4.2.2	让命令行认识 cl	23
4.3	第一个程序：无界面的仿真	24
4.3.1	代码	24
4.3.2	用命令行编译	25
4.4	可视化程序	26
4.4.1	可视化管线的两段式结构	26
4.4.2	GLFW 从哪里来	26
4.4.3	最小可视化程序	26
4.4.4	常用交互控制	29
4.5	DLL 部署	29
第五章	CMake 工程配置	31
5.1	安装 CMake	31
5.1.1	三种来源	31

5.1.2	生成器怎么选	31
5.2	先确认 SDK 提供了什么	32
5.3	用法一: find_package (推荐)	32
5.3.1	导入目标的名字	32
5.3.2	告诉 CMake 去哪里找	33
5.4	完整模板 A: find_package 版	33
5.5	完整模板 B: 手工导入库版	35
5.6	构建与运行	35
5.6.1	Visual Studio 生成器 (多配置)	35
5.6.2	Ninja 生成器 (单配置)	36
5.6.3	在 Visual Studio 里打开	36
5.7	CMakePresets.json: 把命令行固化下来	36
5.8	路线三: vcpkg 与源码安装	37
5.8.1	vcpkg	37
5.8.2	从源码安装到自己的目录	38
5.9	排错表	39
第六章	从源码编译 MuJoCo (可选)	40
6.1	什么时候需要自己编译	40
6.2	准备工作	41
6.3	配置、构建与安装	41
6.4	常用 CMake 选项	42
6.5	验证构建结果	43
6.6	常见编译失败	43
第七章	SolidWorks 建模与模型导出	44
7.1	先明确 MuJoCo 要什么	44
7.2	SolidWorks 侧的六个建模约定	44
7.2.1	单位: 毫米建模, 米导出	45
7.2.2	坐标系: 原点放在关节轴线上	45
7.2.3	拆分: 一个连杆一个子装配体	45
7.2.4	命名: 让 XML 里的名字和 CAD 一致	46
7.2.5	质量属性: 材质密度必须真实	46
7.2.6	碰撞体: 单独做简化件	46
7.3	方案 A: 用 SW2URDF 插件导出 URDF	47
7.3.1	获取与安装	47
7.3.2	导出步骤	47
7.3.3	导出产物	48

7.3.4	大装配体的整理技巧	48
7.4	方案 B: 直接导出 OBJ / STL	49
7.5	导出之后: 先量一量	49
7.6	两条路线对照与常见坑	50
第八章	网格处理与 MJCF 模型编写	52
8.1	工程目录组织	52
8.1.1	推荐布局	52
8.1.2	资源搜索规则	53
8.2	网格与材质的写法	54
8.2.1	网格元素	54
8.2.2	外观: 材质与纹理	54
8.3	最小可运行模型	55
8.3.1	逐段解读	56
8.3.2	验证	57
8.4	从 URDF 转成 MJCF	58
8.4.1	直接加载	58
8.4.2	转换时的默认值变化	58
8.4.3	转换后必须手工补的东西	59
8.5	用 obj2mjcf 处理复合网格	59
8.6	碰撞几何	60
8.6.1	碰撞只用凸包	60
8.6.2	视觉与碰撞分离的写法	60
8.7	常见 MJCF 错误	61
第九章	模型验证、调参与排错	62
9.1	先用眼睛看一遍	62
9.2	量化自检	62
9.2.1	第一条: 质量与惯量	62
9.2.2	第二条: 警告计数	63
9.2.3	第三条: 静态平衡检查	64
9.3	稳定性与调参	65
9.3.1	参数速查	65
9.3.2	按症状查方向	66
9.4	报错信息索引	66
9.5	性能优化	66

附录 A	速查表与常见问题	68
A.1	工程目录模板	68
A.2	环境变量与路径速查	68
A.3	命令速查	69
A.4	版本与默认值速查	70
A.5	MJCF 元素速查	71
A.6	资源	71
A.7	常见问题	71

第一章 MuJoCo 简介与 Windows 环境准备

MuJoCo (*Multi-Joint dynamics with Contact*) 是一个面向多关节刚体系统与接触的通用物理引擎。它最初由华盛顿大学的 Emo Todorov 开发，用于神经科学与机器人控制的仿真研究；2021 年 10 月起以 Apache 2.0 许可完全开源，现由 Google DeepMind 维护，是强化学习、机器人控制、生物力学与最优控制领域使用最广的仿真引擎之一。

本指南面向 **Windows** 平台，覆盖三条主线：**Python** 端调用（第 2、3 章）、**C++** 端调用与 CMake 工程配置（第 4、5、6 章）、以及把 **SolidWorks** 中的 CAD 模型导入 MuJoCo（第 7、8 章）。全书以 **MuJoCo 3.14.0**（2026-09-22 发布）为版本基准。

1.1 引擎定位与设计特点

MuJoCo 求解的动力学方程为

$$M(q)\ddot{q} + c(q, \dot{q}) = \tau + J^T f \quad (1.1)$$

其中 q 为广义坐标， $M(q)$ 为质量矩阵， $c(q, \dot{q})$ 为科氏力、离心力与重力项之和， τ 为广义力（执行器与关节阻尼）， J 为接触点的雅可比矩阵， f 为接触力。与多数通用物理引擎不同，MuJoCo 的接触力 f 不是由碰撞冲量逐点求解，而是由带约束的凸优化一次性求出，这也是它数值稳定、适合做最优控制与强化学习的原因。

定义 1.1（广义坐标） 系统的位形用广义坐标 $q \in \mathbb{R}^{n_q}$ 描述，速度用 $\dot{q} \in \mathbb{R}^{n_v}$ 描述。对只有转动与移动关节的机械臂有 $n_q = n_v$ ；对含自由关节（浮基）与四元数的系统， $n_q > n_v$ ，此时 $\dot{q}$ 与 q 的维数不同，编程时必须区分 `model.nq` 与 `model.nv`。

引擎的四个关键设计如下：

1. **软约束**：接触、关节限位、等式约束都以「弹簧-阻尼」形式软化处理，避免刚性约束带来的求解困难，参数用 `solref`、`solimp` 调节。
2. **隐式积分**：阻尼与部分力项隐式处理，配合可选的 `implicitfast` 积分器，允许比显式积分更大的时间步长（默认 `timestep=0.002 s`）。

3. **MJCF 模型格式**: 基于 XML 的场景描述语言，可直接手写，也可由 URDF、SolidWorks 模型转换或程序化生成。
4. **一套引擎两套接口**: Python 绑定与 C 语言 API 共用同一份求解器代码，模型文件与数值结果完全一致。

表 1.1: MuJoCo 与常见仿真器的定位对比

引擎	模型格式	许可证	定位与典型用途
MuJoCo	MJCF (可读 URDF)	Apache 2.0	接触丰富、需要快速稳定仿真的场合: 强化学习环境、机械臂与足式机器人控制、最优控制
PyBullet	URDF / SDF / MJCF	zlib	上手快、生态偏 RL, 接触精度与求解速度不如 MuJoCo
Isaac Sim / PhysX	USD	专有 (免费)	GPU 大规模并行、渲染逼真, 硬件要求高、部署重
Gazebo	SDF / URDF	Apache 2.0	ROS 生态的完整机器人仿真 (传感器、插件、通信), 实时性一般
Drake	URDF / SDF	BSD-3	侧重优化与验证的机器人工具箱, 学习曲线较陡

版本与兼容性 MuJoCo 3.x 的次版本号更新较快 (3.14.0 于 2026-09-22 发布)。小版本之间不保证 **API 完全兼容**: 例如 3.14.0 修改了 `mj_readCtrl` 的函数签名, 并使 `spec` 的 `signature` 语义发生变化。升级版本后若编译报错, 先查官方 Release Notes 的 *Breaking API changes* 一节。

1.2 三种使用方式

同一份引擎有三种用法, 选择取决于你要把仿真嵌到哪里 (图 1.1)。

表 1.2: MuJoCo 的三种使用方式

方式	安装	需要的工具链	适用场合
Python 绑定	<code>pip install mujoco</code>	无 (wheel 自带引擎与渲染后端)	算法验证、强化学习、数据处理、教学; 推荐所有人先走这条路
C++ 预编译库	下载 <code>mujoco-3.14.0-windows-x86_64.zip</code>	Visual Studio 2022 + CMake	把仿真嵌进自己的 C++ 程序: 实时控制、自研求解器、无 Python 的部署环境
C++ 源码编译	<code>git clone</code> (依赖由 FetchContent 拉取)	同上 + Git + 网络	需要修改引擎、自定义渲染、给 MuJoCo 提交代码

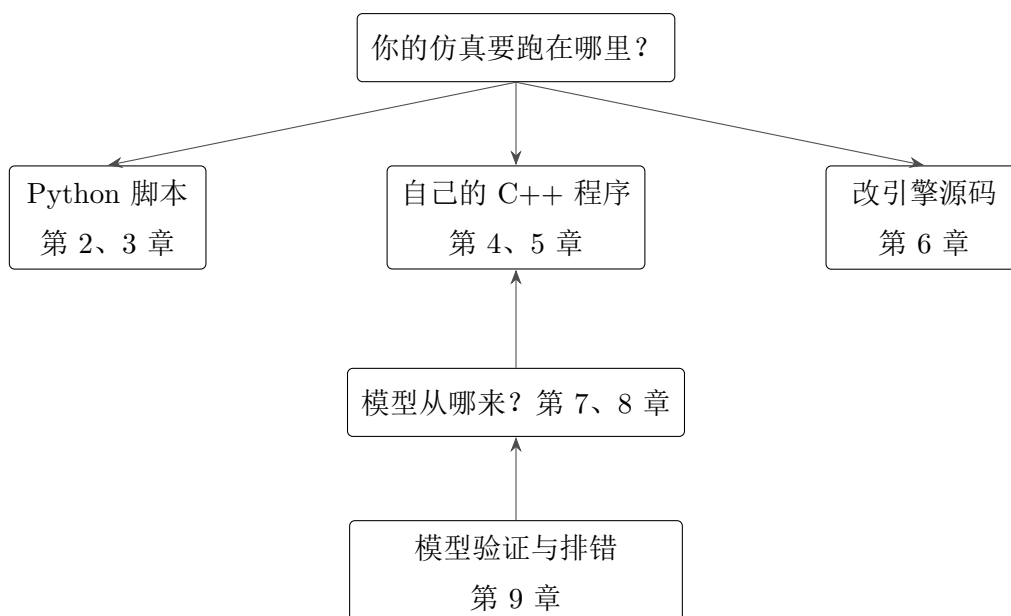


图 1.1: 按用途选择阅读路线

注：三条路线并不互斥。实际项目里常见「Python 里调模型、C++ 里跑控制器」的组合，此时两边加载的是同一个 MJCF 文件，只要引擎版本一致，模型编译结果就一致。

1.3 Windows 环境要求

表 1.3: 硬件与系统要求

项目	要求	说明
操作系统	Windows 10 1809 及以上 / Windows 11	仅 64 位；官方不提供 32 位与 ARM 版 Windows 构建
CPU	x86_64，支持 SSE2	引擎为单线程 C 代码，主频与单核性能决定仿真速度；大规模并行要靠多进程而非多线程
显卡	支持 OpenGL 1.5 与 ARB_framebuffer_object、ARB_vertex_buffer_object 扩展的显卡驱动	仅交互式可视化需要。纯计算（不加界面地跑 mj_step）不需要显卡，也不需要 OpenGL
内存	8 GB	复杂场景与大规模并行采样建议 16 GB 以上
磁盘	约 1 GB	预编译包约 22 MB，Python wheel 约 60 MB，另加编译工具链

远程桌面与虚拟机 通过 Windows 远程桌面 (RDP) 连接时, 系统报告的 OpenGL 版本可能被降级到 1.1, 导致 `simulate` 或 `mujoco.viewer` 启动时报 `GLFW error: WGL: The driver does not appear to support OpenGL`。解决办法: 在本机运行界面, 或改用无界面模式 (Python 端可用 `mujoco.osmesa` 后端做离屏渲染, 详见 3.4 节), 或使用支持 OpenGL 直通的远程方案。

1.4 软件清单与安装顺序

表 1.4: 本指南涉及的软件

软件	建议版本	用途	必需性
Python	3.10 ~ 3.14	Python 端仿真与模型转换	走 Python 路线时必需
MuJoCo wheel	3.14.0	<code>pip install mujoco</code> 自动安装	同上
Visual Studio 2022	17.x (v143 工具集)	C++ 编译器与调试器	走 C++ 路线时必需, 安装时勾选「使用 C++ 的桌面开发」
CMake	3.24 及以上	生成与构建 C++ 工程	走 C++ 路线时必需
MuJoCo 预编译包	3.14.0 (<code>windows-x86_64</code>)	C++ 的头文件、导入库与 DLL	走 C++ 路线时必需
Git	任意近期版本	拉取 MuJoCo 源码与示例模型	仅源码编译与克隆模型库时需要
SolidWorks	2020 及以上	提供 CAD 模型; SW2URDF 插件导出 URDF	仅第 7 章需要
TeX Live	2025	编译本笔记 (XeLaTeX)	仅需要重新编译本文档时

推荐按下面的顺序安装, 每装完一项立即做一次自检, 避免问题堆积:

1. 装 Python, 并在命令行确认 `python --version`;
2. `pip install mujoco`, 确认 `python -c "import mujoco"` (见 2.4 节);
3. (走 C++ 路线) 装 Visual Studio 2022, 确认 `cl` 可用;
4. 装 CMake, 确认 `cmake --version`;
5. 解压 MuJoCo 预编译包, 编译官方示例, 确认能出现可视化窗口。

1.5 目录规划与环境自检

Windows 上的路径问题绝大多数来自三处：中文路径、空格路径、过长的路径。建议把工具链与模型放在一个短路径下，例如：

```
1 D:\mujoco\                (root: toolchain and SDK)
2 |-- mujoco-3.14.0\        (extracted release archive)
3 |   |-- bin\              (mujoco.dll, simulate.exe, ...)
4 |   |-- doc\              (README.txt, REFERENCE.txt)
5 |   |-- include\mujoco\   (headers needed to build)
6 |   |-- model\            (bundled MJCF models)
7 |   |-- sample\           (C++ samples + CMakeLists.txt)
8 |   `-- lib\              (import libs; check your archive)
9 `-- projects\
10     |-- my_sim\           (your own project)
11         |-- CMakeLists.txt
12         |-- main.cc
13         `-- model\
```

官方文档给出的目录说明只有五项：bin（动态库、可执行文件、MUJOCO_LOG.TXT）、doc（说明与 API 参考）、include（头文件）、model（模型库）、sample（示例与 CMakeLists.txt）。MSVC 链接需要导入库（mujoco.lib），因此实际压缩包里通常还会有一个 lib 目录或等价的 CMake 包配置——解压后请先看一眼实际有哪些目录，第五章会按「有 CMake 包配置」和「只有头文件与导入库」两种情况分别给出写法。

环境自检：打开一个新的 PowerShell 窗口，逐条执行下列命令，确认都能正常输出（cl 需要「x64 Native Tools Command Prompt for VS 2022」或已执行过 vcvars64.bat）：

```
1 python --version          # expect: Python 3.10 - 3.14
2 python -c "import mujoco; print(mujoco.__version__)" # expect: 3.14.0
3 cmake --version           # expect: 3.24 or later
4 cl                         # expect: Microsoft (R) C/C++ Optimizing
                             Compiler 19.3x
5 echo %MUJOCO_DIR%         # if you set the SDK path manually, it echoes
                             back
```

为什么要用短路径 MuJoCo 通过 XML 中的相对路径寻找网格与纹理文件，路径中一旦出现中文或空格，mj_loadXML 在部分版本与编码环境下会报 Error: could not open file。把模型放在纯英文短路径下可以从根本上绕开这一类问题。

第二章 Python 环境与 mujoco 库安装

Python 路线是上手 MuJoCo 最快的路径：官方提供的 mujoco 包自带你所需的引擎动态库，不需要另外下载 SDK，也不需要编译器。本章只解决「把环境装对、装干净」这一件事，具体怎么用见第三章。

2.1 Python 版本选择与安装

2.1.1 版本要求

MuJoCo 3.14.0 的官方 wheel 覆盖 **Python 3.10 ~ 3.15**，且只有 **64 位 Windows** (`win_amd64`) 的构建，没有 32 位版本。实践中的选择建议见表 2.1。

表 2.1: Python 版本选择建议

版本	评价	说明
3.10 / 3.11	可用	部分老牌强化学习库仍以它们为主要测试版本
3.12 / 3.13	推荐	与 NumPy、PyTorch、gymnasium 等生态的兼容性最稳
3.14 / 3.15	可用	mujoco 官方已出 wheel，但周边库（尤其是深度学习框架）可能还没跟上

2.1.2 三种安装方式

1. **python.org 官方安装包**（推荐）：下载 Windows installer (64-bit)，安装时务必勾选 *Add python.exe to PATH*，并在 *Customize installation* 中确认勾选 `pip` 与 `tcl/tk`。
2. **Miniconda / Anaconda**：注意 conda 渠道里也有一个叫 mujoco 的包（`conda install -c conda-forge mujoco`），它装出来的是 C/C++ 库与仿真程序，不含 Python 绑定；Python 绑定请仍然用 `pip install mujoco`。
3. **Microsoft Store 版**：不推荐。它的安装路径与权限模型和普通 Python 不同，容易与 `pip` 的脚本目录冲突。

安装路径不要带中文与空格 把 Python 装在 D:\Python312 这类纯英文短路径下。装在「C:\Users\张三\我的软件\Python」下时，pip 的脚本、.pth 文件与部分编译型包都会出现难以定位的路径问题。

2.1.3 验证 Python 与多版本共存

```
1 python --version
2 where python
3 pip --version
4 py -0
```

其中 `py -0` 会列出本机安装的所有 Python 版本（`py` 是 Windows 自带的版本启动器）。若装了多个版本，可以显式指定解释器来创建环境，例如 `py -3.12 -m venv .venv`。

2.2 虚拟环境

2.2.1 为什么必须用

`pip install mujoco` 默认装进当前解释器的 `site-packages`。不加隔离时，不同项目对 NumPy、PyTorch 的版本要求会互相打架，而 `mujoco` 又恰好会连带安装 `numpy`、`absl-py`、`etils`、`glfw`、`pyopengl` 等一批依赖。每个项目一个虚拟环境是成本最低的隔离方式。

2.2.2 conda: 跨语言环境

```
1 conda create -n mujoco python=3.12 -y
2 conda activate mujoco
3 python -m pip install mujoco
```

2.3 安装 mujoco

2.3.1 基本安装

在已激活的虚拟环境中：

```
1 pip install mujoco
2 pip install mujoco==3.14.0
3 pip install --upgrade mujoco
4 pip index versions mujoco
```

四条命令依次是：装最新版、装指定版本、升级、查询可用版本。要做可复现的实验，建议在项目里固定版本，并导出依赖清单：

```
1 pip freeze > requirements.txt
2 pip install -r requirements.txt
```

2.3.2 国内镜像加速

清华 TUNA 镜像适合一次性使用：

```
1 pip install mujoco -i https://pypi.tuna.tsinghua.edu.cn/simple
```

也可以永久写入配置（写入 `~/pip/pip.ini`，即 `%APPDATA%\pip\pip.ini`）：

```
1 pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
2 pip config set global.trusted-host pypi.tuna.tsinghua.edu.cn
3 pip config list
```

内网与离线机器 先在一台能联网的同版本机器上把 wheel 下载下来，再拷进内网安装：`pip download mujoco -d .\wheels` 与 `pip install --no-index --find-links .\wheels mujoco`。注意 mujoco 的 wheel 是与 Python 版本、平台绑定的，下载机的 Python 次版本必须与目标机一致。

2.3.3 不要装 mujoco-py

mujoco-py 已废弃 mujoco-py 是 OpenAI 为旧版 MuJoCo 2.1 写的第三方封装，依赖已失效的许可证文件与旧版引擎，不要安装它。网上大量「`pip install mujoco-py` 报错」的教程都已过时。判断方法：mujoco-py 需要 MJKEY 或 `~/.mujoco/mjkey.txt`，而现代的 mujoco 包不需要任何密钥。

2.4 安装验证

2.4.1 四步自检

```
1 python -c "import mujoco, sys; print('mujoco', mujoco.__version__); print('python', sys.version)"
2 python -c "import mujoco; print('header', mujoco.mjVERSION_HEADER)"
3 python -c "import mujoco, numpy; print(numpy.__version__)"
4 python -c "import mujoco; print(mujoco.__file__)"
```

第三条确认 NumPy 可用，第四条确认当前解释器加载的到底是哪一个 mujoco（排除「装到了另一个 Python 里」）。前两条的输出应当满足「库版本与头文件版本一致」，即 `mujoco.__version__` 对应的整数与 `mjVERSION_HEADER` 相同。

2.4.2 跑通第一个模型

把下面的内容存成 `hello.xml`（本章目录下即可），它是官方文档首页的示例模型：一个固定在世界里的地面、一个光源，以及一个自由下落的方块。

```
1 <mujoco>
2   <worldbody>
3     <light diffuse=".5 .5 .5" pos="0 0 3" dir="0 0 -1"/>
4     <geom type="plane" size="1 1 0.1" rgba=".9 0 0 1"/>
5     <body pos="0 0 1">
6       <joint type="free"/>
7       <geom type="box" size=".1 .2 .3" rgba="0 .9 0 1"/>
8     </body>
9   </worldbody>
10 </mujoco>
```

```
1 import mujoco
2
3 model = mujoco.MjModel.from_xml_path("hello.xml")
4 data = mujoco.MjData(model)
5
6 print("nq =", model.nq, " nv =", model.nv, " nu =", model.nu)
7 print("timestep =", model.opt.timestep)
8
9 while data.time < 1.0:
10     mujoco.mj_step(model, data)
11
12 print("time =", data.time)
13 print("qpos =", data.qpos)
```

期望输出：`nq = 7`、`nv = 6`、`nu = 0`。这正是定义 1.1 中 $n_q > n_v$ 的典型情形——自由关节的位置是 3 个平移加 4 个四元数共 7 个数，速度是 3 个线速度加 3 个角速度共 6 个数。仿真结束后 `qpos[2]` 应当接近方块的半高 0.3（方块落在地面上）。

2.4.3 打开交互窗口

```
1 python -m mujoco.viewer
```

```

2 python -m mujoco.viewer --mjcf=hello.xml
3 python -m mujoco.viewer --mjcf=C:\mujoco\mujoco-3.14.0\model\humanoid\
    humanoid.xml

```

第一条启动空会话，之后可以把 MJCF 文件直接拖进窗口；第二条打开指定模型。窗口内的常用操作：左键拖拽旋转、右键拖拽平移、滚轮缩放、双击物体选中并按住 Ctrl 拖动施加外力、Tab 收起左右面板。

注：上面第二条命令中的 `humanoid.xml` 来自预编译 SDK 的 `model` 目录（见 4.1 节），pip 安装的 `mujoco` 包本身不带这套模型集合。想用官方模型库时，可以克隆 MuJoCo Menagerie (`git clone https://github.com/google-deepmind/mujoco_menagerie`)，或者下载预编译 zip 只取其中的 `model` 目录。

2.5 与周边生态的关系

装好 `mujoco` 之后，往上还有一层「用起来更省事」的库。它们的定位见表 2.2，选择原则是：先直接用 `mujoco`，缺什么再往上加。

表 2.2: Python 生态中的常见搭配

库	安装	定位
<code>mujoco</code>	<code>pip install mujoco</code>	官方底层绑定，直接对应 C API；本指南的主角
<code>mujoco-py</code>	不要安装	已废弃的第三方封装，见 2.3.3 节
<code>gymnasium</code>	<code>pip install "gymnasium[mujoco]"</code>	强化学习环境接口， <code>gym.make("Ant-v5")</code> 即可用 MuJoCo 环境
<code>dm_control</code>	<code>pip install dm_control</code>	DeepMind 的物理与任务库，依赖 <code>mujoco</code> ，含大量现成任务
<code>MJX</code>	<code>pip install mujoco-mjx</code>	MuJoCo 的 JAX 后端，可在 GPU 上批量仿真，适合大规模并行与可微仿真
<code>MuJoCo Warp</code>	见官方文档	NVIDIA Warp 后端，同样是 GPU 批量仿真
<code>mujoco.rollout</code>	随 <code>mujoco</code> 附带	把「循环 <code>mj_step</code> 」搬到 C++ 层并用线程池并行，采样提速明显
<code>mujoco[usd]</code>	<code>pip install mujoco[usd]</code>	把场景与轨迹导出成 USD，交给 Omniverse / Blender 渲染

2.6 常见安装问题

表 2.3: pip 安装阶段常见问题

现象	原因	处理
Could not find a version that satisfies the requirement mujoco	Python 版本不在支持范围，或用了 32 位 Python	用 <code>platform.architecture()</code> 确认是 64 位，且版本在 3.10 ~ 3.15
ERROR: Could not build wheels (开始编译源码)	没有匹配当前 Python 版本的 wheel, pip 退回到源码编译	换用受支持的 Python 次版本；不要试图在 Windows 上从源码装 Python 绑定
下载卡住或超时	直连 PyPI 不稳定	用 3.3.2 节的镜像，或配置代理 <code>pip --proxy http://host:port install mujoco</code>
Access is denied	装进了系统 Python 的 Program Files 目录	在虚拟环境里装，不要用管理员权限的全局安装
import mujoco 报 DLL load failed	加载到了错误或残缺的安装；或混装了多个版本	打印 <code>mujoco.__file__</code> 确认路径；卸载后重装 <code>pip uninstall mujoco -y && pip install mujoco</code>
同时存在 conda 的 mujoco 与 pip 的 mujoco	两套文件互相覆盖	只保留一套：conda 渠道的包用于 C++，Python 绑定用 pip 装进独立环境

下一步 环境就绪后进入第三章：加载模型、推进仿真、读取状态，以及把画面显示出来或渲染成图片。

第三章 Python 基础用法与可视化

第二章把环境装好了，本章解决「怎么用」。MuJoCo 的 Python 绑定是对 C API 的近乎直接的映射：所有函数名、参数顺序都与 C 一致，只有少数地方为了符合 Python 习惯做了调整（例如 `mj_step` 多了一个 `nstep` 参数，数组长度参数被省略）。理解这一点，就能直接查官方 C API 文档来写 Python 代码。

3.1 模型与数据

3.1.1 两个核心对象

MuJoCo 把「不变的模型」与「变化的状态」严格分开：

定义 3.1 (`mjModel`) 模型的编译结果：几何、关节、质量与惯量、执行器、传感器、选项与默认值。它在仿真过程中保持不变，对应 Python 类 `mujoco.MjModel`。

定义 3.2 (`mjData`) 全部随时间变化的量与中间计算结果：广义坐标、速度、加速度、控制量、接触力、雅可比、传感器读数等。运行期不再分配内存，全部预分配在 `MjData` 里。对应 Python 类 `mujoco.MjData`。

这种划分带来两个好处：同一个模型可以创建任意多个数据实例（多线程并行采样），而模型本身可以安全共享。

3.1.2 加载模型

`MjModel` 没有 Python 构造函数，只能通过三个工厂方法创建：

```
1 import mujoco
2
3 m = mujoco.MjModel.from_xml_path("model/robot.xml")
4 m = mujoco.MjModel.from_binary_path("model/robot.mjb")
5 m = mujoco.MjModel.from_xml_string(xml_string, assets_dict)
```

- `from_xml_path`: 从文件加载 MJCF 或 URDF，格式由扩展名与内容自动识别，因此可以直接打开 `robot.urdf`；
- `from_binary_path`: 加载编译好的二进制模型 (`.mjb`)，加载快但不含 XML 信息、且与版本绑定；
- `from_xml_string`: 从字符串加载，第二个参数是可选的「虚拟文件系统」字典，把网格、纹理等内容直接以内存里的字节流提供给编译器——把模型打包进程序或数据库时非常有用，也避免了路径问题。

3.1.3 常用字段

表 3.1: 最常用的模型与数据字段

字段	含义
<code>model.nq</code>	广义坐标个数（位置）
<code>model.nv</code>	自由度个数（速度），无自由关节时 $n_v = n_q$
<code>model.nu</code>	执行器个数，即 <code>data.ctrl</code> 的长度
<code>model.nbody</code> / <code>ngeom</code> / <code>njnt</code> / <code>nsite</code>	物体、几何体、关节、站点的个数
<code>model.opt.timestep</code>	积分步长，默认 0.002 s
<code>model.body_mass</code>	各物体的质量，模型装错时第一个该查的地方
<code>data.time</code>	当前仿真时刻
<code>data.qpos</code>	广义坐标，长度 <code>nq</code>
<code>data.qvel</code>	广义速度，长度 <code>nv</code>
<code>data.qacc</code>	广义加速度，长度 <code>nv</code>
<code>data.ctrl</code>	控制输入，长度 <code>nu</code>
<code>data.act</code>	执行器内部状态（如 <code>pid</code> 、 <code>dcmotor</code> 的内置状态）
<code>data.sensordata</code>	传感器读数，长度由模型决定
<code>data.xpos</code> / <code>data.xquat</code>	各物体在世界坐标系中的位置与姿态（ n_q 的位形经正运动学算出）
<code>data.qfrc_applied</code> / <code>data.xfrc_applied</code>	用户施加的广义力 / 笛卡尔力，用来做外部扰动

数组是「视图」而不是「副本」 `data.qpos` 返回的是指向引擎内存的 NumPy 视图。写 `history.append(data.qpos)` 会得到一个「所有元素都相同」的列表——因为每一步 `mj_step` 都会就地改写这块内存。记录数据时必须 `.copy()`：
`history.append(data.qpos.copy())`。

3.2 推进仿真

3.2.1 三个入口

```
1 mujoco.mj_step(m, d)           # 前进一个时间步（完整的前向动力学 + 积分）
2 mujoco.mj_step(m, d, nstep=20) # Python 专有：连续走 20 步，中途不回到解释器
3 mujoco.mj_forward(m, d)        # 只做前向计算，不推进时间（改完状态后先调它）
4 mujoco.mj_step1(m, d); ...     # 只算到「位置相关」的部分
5 mujoco.mj_step2(m, d)         # 只算到「速度相关」的部分
6 mujoco.mj_resetData(m, d)      # 回到模型的初始状态
```

`mj_step1/mj_step2` 把一步拆成两半，中间是施加控制的正确位置（`mjcb_control` 回调即在此处被调用）。若只是「推进仿真」，用 `mj_step` 即可；想自己写控制器且不装回调，用下面的顺序更清晰：

```
1 mujoco.mj_step1(m, d)
2 d.ctrl[:] = policy(d)          # 或直接改 d.qfrc_applied
3 mujoco.mj_step2(m, d)
```

3.2.2 状态存取与关键帧

```
1 spec = mujoco.mjtState.mjSTATE_FULLPHYSICS
2 n = mujoco.mj_stateSize(m, spec)
3 buf = np.empty(n)
4 mujoco.mj_getState(m, d, buf, spec)    # 存
5 mujoco.mj_setState(m, d, buf, spec)    # 取
6 mujoco.mj_resetDataKeyframe(m, d, 0)   # 回到第 0 个关键帧 <key .../>
```

整状态的存取比逐个变量抄写更安全，做强化学习环境的重置（reset）时尤其有用。若模型里定义了 `<key>`（关键帧），直接用 `mj_resetDataKeyframe` 即可回到一个给定的初始姿态。

3.2.3 一个完整的受控例子

下面给方块加一个滑轨与一个位置执行器，并用简单比例控制把它拉到目标高度。把它和第三章开头的 `hello.xml` 对照着看，可以体会 MJCF 与 Python 两侧的分工：模型里定义「有什么」，程序里定义「怎么用」。

```

1 <mujoco>
2   <option timestep="0.002"/>
3   <worldbody>
4     <light pos="0 0 3" dir="0 0 -1"/>
5     <geom type="plane" size="1 1 0.1"/>
6     <body name="slider" pos="0 0 0.5">
7       <joint name="slide" type="slide" axis="0 0 1"/>
8       <geom type="box" size=".05 .05 .05" rgba="0 .6 .9 1"/>
9     </body>
10  </worldbody>
11  <actuator>
12    <position name="lift" joint="slide" kp="200" kv="20"/>
13  </actuator>
14 </mujoco>

```

```

1 import mujoco
2
3 m = mujoco.MjModel.from_xml_path("slider.xml")
4 d = mujoco.MjData(m)
5
6 target = 0.8
7 act_id = m.actuator("lift").id
8
9 for _ in range(2000):
10     d.ctrl[act_id] = target
11     mujoco.mj_step(m, d)
12
13 print("time =", d.time)
14 print("height =", d.joint("slide").qpos[0])

```

期望输出中 `height` 收敛到 0.8 附近。注意这里用的是 MJCF 自带的位置执行器（内部就是 PD），比自己写 PD 更省事，也更容易复现。

3.3 命名访问

3.3.1 按名字拿数组

MuJoCo 的 C API 用整数 `id` 索引数组，Python 绑定在此基础上提供了「命名访问」：`m.`类别（'名字'）返回一个访问器，属性名就是去掉前缀的字段名，并且访问是 $O(1)$ 的，不随模型规模变慢。

```

1 m.geom("gizmo").rgba          # 长度为 4 的 NumPy 视图，可直接赋值改颜色
2 m.geom("gizmo").id            # 等价于 mujoco.mj_name2id(m, mjOBJ_GEOM, "
    gizmo")
3 m.geom(3).name                # 反向查询
4 d.body("slider").xpos         # 物体的世界坐标
5 d.joint("slide").qpos         # 只属于该关节的那一段 qpos
6 m.joint("slide").range        # 关节限位

```

可用的类别名见表 3.2。关节类别同时接受 `jnt` 与 `joint`，相机接受 `cam` 与 `camera`，其余别名同理。

表 3.2: 命名访问的类别

body	jnt/joint	geom	site	cam/camera
light	mesh	skin	hfield	tex/texture
mat/material	pair	exclude	eq/equality	tendon/ten
actuator	sensor	numeric	text	key/keyframe

工程习惯 凡是程序里要反复引用的元素——关节、执行器、站点、相机——都在 MJCF 里起名字，然后在 Python 里用命名访问取用。这样模型改名或增删部件时，程序里对应的代码不必跟着改 `id`。

3.4 交互式可视化

3.4.1 三种用法

官方在 `mujoco.viewer` 模块里提供三种用法（表 3.3）。它们的区别只在「谁负责推进物理」：managed 模式由查看器自己推进，passive 模式由你的脚本推进。

表 3.3: 三种查看器用法

用法	调用	适用
managed	<code>viewer.launch(model, data)</code>	只看模型、手动拖动交互；会阻塞你的代码
standalone	<code>python -m mujoco.viewer</code> <code>--mjc=...</code>	命令行直接看模型文件，等价于 C++ 的 <code>simulate</code>
passive	<code>viewer.launch_passive(model, data)</code>	自己写控制循环，同时把画面显示出来；不阻塞

3.4.2 passive 查看器

这是做控制与学习最常用的模式。要点有三条：用 `with` 管理生命周期、每步调用 `sync()`、改动物理或可视化状态前先 `lock()`。

```

1 import time
2 import mujoco
3 import mujoco.viewer
4
5 m = mujoco.MjModel.from_xml_path("slider.xml")
6 d = mujoco.MjData(m)
7
8 paused = False
9
10 def key_callback(keycode):
11     global paused
12     if chr(keycode) == " ":
13         paused = not paused
14
15 with mujoco.viewer.launch_passive(m, d, key_callback=key_callback) as
    viewer:
16     start = time.time()
17     while viewer.is_running() and time.time() - start < 30:
18         step_start = time.time()
19
20         if not paused:
21             d.ctrl[0] = 0.8
22             mujoco.mj_step(m, d)
23
24             with viewer.lock():
25                 viewer.opt.flags[mujoco.mjtVisFlag.mjVIS_CONTACTPOINT] =
int(d.time % 2)
26
27             viewer.sync()
28
29             dt = m.opt.timestep - (time.time() - step_start)
30             if dt > 0:
31                 time.sleep(dt)

```

- `sync()` 是「把 `mjModel/mjData` 交给 GUI，再把 GUI 上的改动（鼠标扰动、控制输入、各种开关）取回来」的唯一时机。不调用 `sync` 窗口里的东西不会动。
- `sync(state_only=True)` 只同步状态相关的字段并做一次 `mj_forward`，快得多；但它不会反映你对 `mjModel` 的改动（比如改颜色）。

- `viewer.cam`、`viewer.opt`、`viewer.pert` 分别对应相机、可视化选项与扰动对象，可以直接改。
- 想要「按真实时间跑」，用上面末尾的 `time.sleep` 补足每步耗时；纯追求算得快就把它去掉。
- `key_callback` 收到的是键码，`chr()` 可转成字符；空格暂停是最常用的例子。

平台注意 Windows 与 Linux 上 `launch_passive` 直接在普通 python 下运行即可，但必须在主线程调用。macOS 上则必须用包内附带的 `mjpython` 启动脚本 (`mjpython my_script.py`)，否则会因为系统要求「渲染必须在主线程」而失败。

3.4.3 往画面里加自定义图形

`viewer.user_scn` 是一个完全由你控制的 `mjvScene`，可以在里面追加几何体，用来画轨迹、力箭头、目标点等：

```

1 import numpy as np
2 import mujoco
3
4 with mujoco.viewer.launch_passive(m, d) as viewer:
5     while viewer.is_running():
6         mujoco.mj_step(m, d)
7
8         viewer.user_scn.ngeom = 0
9         mujoco.mjv_initGeom(
10             viewer.user_scn.geoms[0],
11             type=mujoco.mjtGeom.mjGEOM_SPHERE,
12             size=[0.02, 0, 0],
13             pos=0.1 * np.array([1, 0, 2]),
14             mat=np.eye(3).flatten(),
15             rgba=np.array([1.0, 0.2, 0.2, 1.0]),
16         )
17         viewer.user_scn.ngeom = 1
18
19     viewer.sync()
```

3.5 离屏渲染与录像

做论文插图、视觉伺服、或把仿真接到图像模型上时，需要不开窗口地渲染出图像。

3.5.1 基本用法

`mujoco.Renderer` 是对底层 OpenGL 上下文 (`mujoco.GLContext`) 的封装：创建时指定分辨率，之后每帧先 `update_scene` 再 `render`。

```
1 import mujoco
2 import numpy as np
3 from PIL import Image
4
5 m = mujoco.MjModel.from_xml_path("slider.xml")
6 d = mujoco.MjData(m)
7
8 renderer = mujoco.Renderer(m, height=480, width=640)
9 try:
10     for _ in range(500):
11         mujoco.mj_step(m, d)
12
13         renderer.update_scene(d)
14         rgb = renderer.render()
15         print(rgb.shape, rgb.dtype)
16         Image.fromarray(rgb).save("frame.png")
17
18         renderer.enable_depth_rendering()
19         renderer.update_scene(d)
20         depth = renderer.render()
21         print(depth.shape, depth.dtype, depth.min(), depth.max())
22         renderer.disable_depth_rendering()
23 finally:
24     renderer.close()
```

- `update_scene` 可以用 `camera=` 指定模型里定义的相机 (`<camera name="cam"/>`)，也可以传 `scene_option` 控制显示哪些元素：`renderer.update_scene(d, camera="cam", scene_option=opt)`。
- 深度渲染给出的深度值不是「米」，其换算与相机的 `fovy`、`znear`、`zfar` 有关，做视觉任务时建议自己接近远平面做一次线性反解，不要直接当距离用。
- 渲染必须在**同一个线程**里进行：OpenGL 上下文一次只能在一个线程上生效。

3.5.2 Windows 上的后端限制

MuJoCo 的离屏渲染在不同平台依赖不同的 OpenGL 上下文实现：Linux 支持 GLX、OSMesa 与 EGL，而 Windows 与 macOS 使用操作系统自带的 OpenGL 实现，也

就是必须有一个（可以是隐藏的）窗口上下文。因此：

- Windows 上不要照搬 Linux 教程里的 `MUJOCO_GL=egl` 或 `MUJOCO_GL=osmesa`，这两个后端在 Windows 上不可用；
- 无显卡的服务器上跑 Windows 虚拟机时，离屏渲染会因为只有 Microsoft 基础显示驱动（OpenGL 1.1）而失败；此时应改用 Linux 容器 + EGL，或者退回到纯计算（不渲染）。

录像把每帧的 `rgb` 收集起来写入视频即可，例如用 `imageio`：
`imageio.mimsave("run.mp4", frames, fps=60)`。注意每帧都要先推进物理，再 `update_scene` 后 `render`。

3.6 模型转换与保存

建模阶段常常需要「加载一个 URDF、稍作修改、再存成 MJCF」。旧接口是「保存最近一次编译的 XML」：

```
1 import mujoco
2
3 m = mujoco.MjModel.from_xml_path("robot.urdf") # 直接编译 URDF
4 mujoco.mj_saveLastXML("robot_from_urdf.xml") # 保存最近一次编译的 XML
```

新接口围绕 `MjSpec`（与 MJCF 一一对应的可编辑对象）展开，适合在程序里改模型，也适合连同资源一起打包：

```
1 import mujoco
2
3 spec = mujoco.MjSpec.from_file("robot.urdf")
4 spec.modelname = "two_link_arm"
5 model = spec.compile()
6 spec.to_zip("robot.mjz") # XML + 网格 + 纹理 打包成一个文件
7 spec2 = mujoco.MjSpec.from_zip("robot.mjz")
```

`.mjz` 是一个 zip 归档，模型、网格与纹理全在里面，给别人或换机器时不会出现「找不到 mesh 文件」的问题，适合做交付。第四章的 C++ 侧同样可以直接加载 `.mjz`。

3.7 常见运行期问题

表 3.4: Python 运行期常见问题

现象	原因	处理
<code>ValueError: Error: could not open file ...</code>	路径不对, 或 XML 里的 <code>mesh/texture</code> 找不到	用绝对路径试一次; 确认 <code>meshdir</code> 与文件实际位置一致
<code>ValueError: Error: unknown element ...</code>	MJCF 拼写错误	MuJoCo 的解析器报错信息通常给出出错的行号与元素名, 直接照行号看
<code>mujoco.FatalError</code>	引擎的不可恢复错误 (如数值发散后的断言)	先降到更小的时间步长; 检查质量与惯量是否合理
模型加载后什么也看不见	XML 里没有 <code><light></code> , 或缺相机	用 <code>simulate</code> 或 <code>viewer</code> 打开同一文件对比; 补光源
窗口一闪而过或打不开	远程桌面、显卡驱动过旧	参见 1.3 节的说明
打印的状态全都一样	记录了 NumPy 视图而非副本	记住 <code>.copy()</code>
仿真「飞掉」	时间步过大、惯量不合理、初始穿插	见第九章的调参与排错

第四章 C++ 开发环境与第一个程序

Python 路线适合验证算法，但真正要嵌进实时控制系统、要和已有的 C++ 工程共用一个进程、或者要在没有 Python 的机器上部署时，就需要直接用 C/C++ 调 MuJoCo。MuJoCo 的运行时是一个编译器无关的 C 动态库，对外只暴露一个稳定的 C API；用 C++ 写只是更方便，链接的东西完全一样。

4.1 获取预编译库

4.1.1 下载

官方推荐不修改引擎源码的用户直接使用预编译库，理由是「自带的依赖版本经过测试，编译选项也做过性能调优，并且几乎完全自包含——除标准 C 运行时外不需要任何其他库」。Windows 的下载地址为：

```
1 https://github.com/google-deepmind/mujoco/releases/download/3.14.0/mujoco-3.14.0-windows-x86_64.zip
2 https://github.com/google-deepmind/mujoco/releases/download/3.14.0/mujoco-3.14.0-windows-x86_64.zip.sha256
```

压缩包约 22 MB，附带一个 .sha256 文件用于校验完整性：

```
1 Get-FileHash .\mujoco-3.14.0-windows-x86_64.zip -Algorithm SHA256
2 Get-Content .\mujoco-3.14.0-windows-x86_64.zip.sha256
```

两行输出应当一致（比较时忽略大小写）。校验通过后解压到短路径，例如 D:\mujoco\。

CPU 必须支持 AVX MuJoCo 的动态库要求处理器支持 AVX 指令集。2011 年之后的 Intel / AMD 处理器基本都支持；在虚拟机里运行时，需要把宿主机的 AVX 指令集透传给虚拟机，否则加载 DLL 时会直接崩溃。

4.1.2 先跑通官方案例

解压后进入 bin 目录，用自带模型验证库本身是好的：

```

1 cd D:\mujoco\mujoco-3.14.0\bin
2 .\simulate.exe ../model\humanoid\humanoid.xml

```

若出现一个人形机器人并且可以旋转、缩放、拖动，说明运行库、OpenGL 与模型库都正常。`simulate` 是官方交互式查看器，它的源码在 `sample/simulate.cc`，也是自己写可视化程序时最好的参考。

注：程序运行目录下会自动生成 `MUJOCO_LOG.TXT`，里面记录错误与警告信息。程序崩溃或行为异常时，第一件事是打开它看最后几行。

4.2 编译工具链

4.2.1 Visual Studio 2022

安装「Visual Studio 2022 生成工具」或完整版 IDE，安装时必须勾选使用 C++ 的桌面开发工作负载，其中包含 MSVC 编译器（`cl.exe`）与 Windows SDK。版本要求：

表 4.1: C++ 工具链要求

组件	版本要求	说明
MSVC	Visual Studio 2019 或更高 (推荐 2022)	官方文档明确要求；2022 的工具集为 v143
Windows SDK	10.0.22000 及以上	低于该版本会在链接期出问题（对应官方 issue #862）
C++ 标准	C++17	头文件与 CMake 配置都按 C++17 编写
CMake	3.24 及以上（官方最低 3.16）	见第五章

4.2.2 让命令行认识 cl

`cl.exe` 不在系统 PATH 里，必须先进入工具链环境。两种做法：

1. 从开始菜单打开 **x64 Native Tools Command Prompt for VS 2022**，该窗口已经配置好全部环境变量；
2. 在普通 PowerShell 里手动引入：

```

1 & "C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Auxiliary\
  Build\vcvars64.bat"
2 cl

```

路径中的 `Community` 按实际安装的版本替换（`Community` / `Professional` / `Enterprise` / `BuildTools`）。`cl` 能打印版本号即表示环境就绪。

官方性能提示 官方文档指出：在 Windows 上用 Clang 编译比用 MSVC 性能更好，官方发布版也是用 Clang 构建的。若对仿真速度敏感，可以安装 LLVM 后用 clang-cl 工具链（CMake 下 -T ClangCL），但先用 MSVC 把流程跑通，再考虑换编译器。

4.3 第一个程序：无界面的仿真

先不碰界面，用最简短的代码确认「能编译、能链接、能加载模型」。

4.3.1 代码

```
1  #include <stdio>
2  #include <mujoco/mujoco.h>
3
4  int main() {
5      // version guard: header and library must match
6      if (mjVERSION_HEADER != mj_version()) {
7          std::printf("version mismatch: header %d, library %d\n",
8                      mjVERSION_HEADER, mj_version());
9          return 1;
10     }
11
12     // load and compile the model
13     char error[1000] = "could not load binary model";
14     mjModel* m = mj_loadXML("hello.xml", nullptr, error, 1000);
15     if (!m) {
16         std::printf("load error: %s\n", error);
17         return 1;
18     }
19
20     // allocate data
21     mjData* d = mj_makeData(m);
22
23     // simulate 10 seconds
24     while (d->time < 10.0) {
25         mj_step(m, d);
26     }
27     std::printf("time = %.3f  qpos[2] = %.4f\n", d->time, d->qpos[2]);
28 }
```

```

29  // free resources, in reverse order
30  mj_deleteData(d);
31  mj_deleteModel(m);
32  return 0;
33  }

```

这段代码里只有四个函数，却构成了所有 MuJoCo 程序的主干：

- `mj_loadXML`：解析并编译模型，返回 `mjModel*`；失败时返回 `nullptr`，错误信息写进 `error` 缓冲区。第二个参数是虚拟文件系统，不需要时传 `nullptr`。
- `mj_makeData`：按模型规模分配 `mjData`，此后运行期不再分配内存。
- `mj_step`：推进一个时间步。
- `mj_deleteData` / `mj_deleteModel`：释放资源。二者的顺序不能颠倒。

版本一致性检查 官方建议在程序里断言头文件版本与库版本一致：`if (mjVERSION_HEADER != mj_version()) complain();`。头文件与 DLL 来自不同版本时，编译器与链接器不一定报错，但运行结果可能莫名其妙——这一行检查能省下大量排查时间。

4.3.2 用命令行编译

假设头文件在 `include`、导入库在 `lib`、DLL 在 `bin`（请按实际解压结果调整；若压缩包里没有 `lib` 目录，见 4.5 节）：

```

1  cl /nologo /std:c++17 /EHsc /O2 /I D:\mujoco\mujoco-3.14.0\include ^
2    hello.cc /link /LIBPATH:D:\mujoco\mujoco-3.14.0\lib mujoco.lib

```

编译成功后得到 `hello.exe`。直接双击运行会提示找不到 `mujoco.dll`，因此运行前把 `bin` 目录加进当前窗口的 `PATH`：

```

1  set PATH=D:\mujoco\mujoco-3.14.0\bin;%PATH%
2  hello.exe

```

期望输出形如 `time = 10.000 qpos[2] = 0.3000`：方块下落后静止在地面上。

注：上面的 `/link /LIBPATH:...` 只是把库目录告诉链接器。工程一旦不止一个源文件，就应该改用第五章的 CMake 写法——手写 `cl` 命令很快会难以维护。

4.4 可视化程序

4.4.1 可视化管线的两段式结构

MuJoCo 的渲染分成两级，理解这一点是看懂示例代码的关键：

1. **抽象可视化(mjv)**: 把 `mjModel` 与 `mjData` 变成一串「抽象的几何体」存进 `mjvScene`，并提供相机、鼠标交互的抽象接口。这一层不依赖任何图形库，换渲染引擎时可以只替换下一层。
2. **OpenGL 渲染 (mjr)**: 把 `mjvScene` 画出来，还提供二维文字、矩形、图像读写等辅助功能。它需要一个已经创建并置为当前的 OpenGL 上下文。

使用 `mjr_` 系列函数就需要一个窗口库来创建 OpenGL 上下文。官方示例用 GLFW，本指南同样如此。

4.4.2 GLFW 从哪里来

官方预编译包里没有给用户工程使用的 GLFW CMake 目标，需要自己准备，三种方式任选：

表 4.2: 获取 GLFW 的三种方式

方式	做法	说明
vcpkg	<code>vcpkg install glfw3</code>	与 CMake 配合最省事，目标名 <code>glfw</code> ；见 5.5 节
官方预编译包	下载 <code>glfw-3.4.bin.WIN64.zip</code>	自带 <code>include/GLFW</code> 、 <code>lib-vc2022/glfw3.lib</code> 与 <code>glfw3.dll</code>
随 MuJoCo 示例一起构建	直接构建 <code>sample</code> 目录	示例的 CMake 会用 <code>FetchContent</code> 自动拉取 GLFW 源码；需要网络与 Git

4.4.3 最小可视化程序

下面的代码是官方样例 `sample/basic.cc` 的骨架（去掉部分鼠标细节），完整版会随预编译包一起发布，建议对照阅读。

```
1 #include <stdio>
2 #include <cstring>
3 #include <GLFW/glfw3.h>
4 #include <mujoco/mujoco.h>
5
6 mjModel* m = nullptr; // model
7 mjData* d = nullptr; // data
```

```

8  mjvCamera cam;           // abstract camera
9  mjvOption opt;           // visualization options
10 mjvScene scn;            // abstract scene
11 mjContext con;           // GPU context
12
13 // keyboard: backspace resets the simulation
14 void keyboard(GLFWwindow* window, int key, int, int act, int) {
15     if (act == GLFW_PRESS && key == GLFW_KEY_BACKSPACE) {
16         mj_resetData(m, d);
17         mj_forward(m, d);
18     }
19 }
20
21 int main(int argc, const char** argv) {
22     if (argc != 2) {
23         std::printf("usage: demo modelfile\n");
24         return 1;
25     }
26
27     // 1. load and compile the model
28     char error[1000] = "could not load model";
29     m = mj_loadXML(argv[1], nullptr, error, 1000);
30     if (!m) {
31         std::printf("load error: %s\n", error);
32         return 1;
33     }
34     d = mj_makeData(m);
35
36     // 2. create a window with an OpenGL context and make it current
37     if (!glfwInit()) {
38         std::printf("could not initialize GLFW\n");
39         return 1;
40     }
41     GLFWwindow* window = glfwCreateWindow(1200, 900, "MuJoCo demo", nullptr,
42         nullptr);
43     glfwMakeContextCurrent(window);
44     glfwSwapInterval(1);           // vsync
45
46     // 3. initialize visualization structures and upload GPU resources

```

```

46  mjv_defaultCamera(&cam);
47  mjv_defaultOption(&opt);
48  mjv_defaultScene(&scn);
49  mjr_defaultContext(&con);
50  mjv_makeScene(m, &scn, 2000);           // capacity: 2000 abstract
    geoms
51  mjr_makeContext(m, &con, mjFONTSCALE_150);
52
53  glfwSetKeyCallback(window, keyboard);
54
55  // 4. main loop: simulate 1/60 s, then render one frame
56  while (!glfwWindowShouldClose(window)) {
57      mjtNum simstart = d->time;
58      while (d->time - simstart < 1.0 / 60.0) {
59          mj_step(m, d);
60      }
61
62      mjrRect viewport = {0, 0, 0, 0};
63      glfwGetFramebufferSize(window, &viewport.width, &viewport.height);
64
65      mjv_updateScene(m, d, &opt, nullptr, &cam, mjCAT_ALL, &scn);
66      mjr_render(viewport, &scn, &con);
67
68      glfwSwapBuffers(window);
69      glfwPollEvents();
70  }
71
72  // 5. teardown, in reverse order
73  mjv_freeScene(&scn);
74  mjr_freeContext(&con);
75  mj_deleteData(d);
76  mj_deleteModel(m);
77  glfwTerminate();
78  return 0;
79 }

```

几个顺序不能换的地方，是初学阶段最容易踩的坑：

1. `glfwMakeContextCurrent` 必须在 `mjr_makeContext` 之前：后者要把网格与纹理上传到 GPU，没有当前上下文就没有 GPU 可用。

2. 每帧的顺序是 `mjv_updateScene` → `mjr_render`；前者根据可视化选项决定「画什么」，后者负责「怎么画」。
3. 退出时先 `mjv_freeScene`、`mjr_freeContext`（释放 GPU 资源），再删模型与数据，最后 `glfwTerminate`。
4. 帧缓冲尺寸用 `glfwGetFramebufferSize` 而不是 `glfwGetWindowSize`：在高分屏缩放的系统上两者不相等，用错会导致画面只画在左下角。反过来，鼠标位移归一化要用窗口尺寸。

4.4.4 常用交互控制

`simulate` 里那些顺手的操作，原理都对应一个函数，自己写时按需接上即可：

表 4.3: 相机与扰动的常用接口

接口	作用
<code>mjv_moveCamera</code>	鼠标钩子：旋转、平移、缩放；配合 <code>mjtMouse</code> 的动作枚举使用
<code>mjv_select</code>	由窗口坐标反查被点中的几何体，用于「点选物体」
<code>mjv_movePerturb</code> 、 <code>mjv_applyPerturbForce</code>	施加外部扰动力，即按住 Ctrl 拖动效果
<code>mjv_defaultPerturb</code>	初始化扰动对象
<code>mjv_updateCamera</code>	只更新相机、不重建几何体列表（相机快速移动时更省）

4.5 DLL 部署

Windows 上「编译通过但一运行就报错」绝大多数是动态库找不到。三种处理方式：

1. **临时：**在当前命令行窗口里 `set PATH=<SDK>\bin;%PATH%`（只对该窗口有效）；
2. **随程序分发：**把 `mujoco.dll`（若使用 GLFW 且它是动态链接的，还有 `glfw3.dll`）拷到 `.exe` 所在目录；
3. **自动拷贝：**在 CMake 里用 `POST_BUILD` 自定义命令，构建后自动把 DLL 复制到输出目录——这是工程化的推荐做法，写法见 5.7 节。

表 4.4: 编译与运行阶段常见问题

现象	原因	处理
Cannot open include file: 'mujoco/mujoco.h'	头文件目录没给对	检查 /I 或 CMake 里的 <code>target_include_directories</code> 指向的是 <code>include</code> 这一层
LNK2019 unresolved external symbol 找不到 mujoco.dll	没链接导入库，或架构不一致	确认链接了 <code>mujoco.lib</code> ；确认 x64 与 SDK 的 64 位一致
0xc000007b	DLL 不在搜索路径	见本节前三条
Cannot open include file: 'GLFW/glfw3.h'	32/64 位混用	统一用 x64 工具链重新编译
undefined reference to <code>glfwInit</code>	缺 GLFW	见 4.4.2 节
运行瞬间退出、没有输出	GLFW 的链接名不对	MSVC 下是 <code>glfw3.lib</code> ；MinGW 下是 <code>libglfw3.a</code> 或 <code>glfw3dll</code>
	模型路径不对或 XML 报错	打开同目录的 <code>MUJOCO_LOG.TXT</code>

下一步 单文件的手写 `cl` 命令到此为止。第五章把工程交给 CMake 管理：找库、链接、自动拷贝 DLL、多配置构建，一次配置好，之后只写业务代码。

第五章 CMake 工程配置

第四章结尾提到，手写 `cl` 命令很快就会难以维护：源文件变多、需要 Debug 与 Release 两套配置、要链接 GLFW、要把 DLL 复制到输出目录……这些正是 CMake 的职责。本章给出可以直接复制使用的完整配置，并解释每一行的作用。

5.1 安装 CMake

5.1.1 三种来源

表 5.1: CMake 的获取方式

方式	说明	建议
官网安装包	下载 <code>cmake-*-windows-x86_64.msi</code> ，安装时勾选 <i>Add CMake to the system PATH</i>	推荐 ，版本最新
winget	<code>winget install Kitware.CMake</code>	适合习惯命令行的用户
Visual Studio 自带	随「使用 C++ 的桌面开发」一起安装	版本往往偏旧，遇到「CMake 版本过低」再单独装新版

```
1 cmake --version
2 where cmake
```

若 `where cmake` 列出多个路径，说明同时存在多份 CMake，注意实际生效的是第一个。

5.1.2 生成器怎么选

CMake 本身不编译代码，它生成「另一套构建系统」的工程文件。Windows 上常用两种：

表 5.2: 常用生成器

生成器	配置数	特点
Visual Studio 17 2022	多配置	生成的 .sln 可用 VS 打开调试； 每个配置单独构建 ，必须用 <code>--config</code> 指定
Ninja	单配置	构建最快；需要在「x64 Native Tools 命令提示符」里运行，配置时用 <code>-DCMAKE_BUILD_TYPE</code> 指定配置

初学者建议先用 Visual Studio 17 2022：出错时可以直接在 IDE 里看调用栈。

5.2 先确认 SDK 提供了什么

MuJoCo 的预编译包能否被 `find_package` 直接找到，取决于包里面有没有 CMake 包配置文件。解压后先看一眼：

```

1 Get-ChildItem D:\mujoco\mujoco-3.14.0 -Depth 1 | Select-Object FullName
2 Get-ChildItem D:\mujoco\mujoco-3.14.0 -Recurse -Filter *Config.cmake |
   Select-Object FullName
3 Get-ChildItem D:\mujoco\mujoco-3.14.0 -Recurse -Filter mujoco*.lib |
   Select-Object FullName

```

根据第二条命令的结果分两种写法（表 5.3）：

表 5.3: 两种 SDK 形态与对应写法

形态	判据	写法
带 CMake 包配置	找到 <code>mujocoConfig.cmake</code> 或 <code>mujoco-config.cmake</code>	用 5.3 节的 <code>find_package(mujoco) + mujoco::mujoco</code>
只有头文件与导入库	有 <code>include/mujoco</code> 、 <code>lib/mujoco.lib</code> 、 <code>bin/mujoco.dll</code>	用 5.4 节手工声明一个 <code>IMPORTED</code> 目标

注：官方文档对预编译包只列出了 `bin`、`doc`、`include`、`model`、`sample` 五个目录，没有明确写 `lib` 与 CMake 配置文件；而 MSVC 链接必须有导入库。因此以你解压后看到的实际内容为准——这两种情况本指南都给了完整写法，不必纠结。

5.3 用法一：find_package（推荐）

5.3.1 导入目标的名字

MuJoCo 的 CMake 包（config 模式）导出的目标是：

表 5.4: MuJoCo 导出的 CMake 目标

目标	含义
<code>mujoco::mujoco</code>	引擎共享库，99% 的情况下只需要它
<code>mujoco::libmujoco_simulate</code>	官方交互式查看器的静态库（由 <code>simulate</code> 子项目导出）
<code>mujoco::filament</code>	只在用 <code>MUJOCO_USE_FILAMENT=ON</code> 构建时存在

没有 `mujoco::glfw` 这个目标 这是一个高频错误: MuJoCo 不导出名为 `mujoco::glfw` 的目标。GLFW 是它自己的依赖，官方示例链接的写法是 `target_link_libraries(basic mujoco::mujoco glfw Threads::Threads)`——注意 `glfw` 是不带命名空间的目标名。

5.3.2 告诉 CMake 去哪里找

`find_package` 在 `config` 模式下会按标准规则搜索 `< 前缀 >/lib/cmake/mujoco/` 一类的目录。SDK 不在系统默认位置时，用下面任一方式指路：

```
1 cmake -S . -B build -Dmujoco_DIR="D:/mujoco/mujoco-3.14.0/lib/cmake/mujoco"
   "
2 cmake -S . -B build -DCMAKE_PREFIX_PATH="D:/mujoco/mujoco-3.14.0"
```

`mujoco_DIR` 要指向含有 `mujocoConfig.cmake` 的那个目录；`CMAKE_PREFIX_PATH` 指向安装前缀，CMake 会自己往下找。路径统一用正斜杠 `/` 或双反斜杠，避免转义问题。

5.4 完整模板 A: `find_package` 版

工程目录：

```
1 my_sim/
2 |-- CMakeLists.txt
3 |-- main.cc
4 `-- model/
5     `-- hello.xml
```

CMakeLists.txt:

```
1 cmake_minimum_required(VERSION 3.24)
2 project(my_sim LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

```

6
7 # ---- MuJoCo ----
8 find_package(mujoco CONFIG REQUIRED)
9
10 # ---- GLFW (只在需要可视化时) ----
11 # find_package glfw3 3.3 CONFIG REQUIRED)
12
13 # ---- 可执行文件 ----
14 add_executable(demo main.cc)
15 target_link_libraries(demo PRIVATE mujoco::mujoco)
16 # target_link_libraries(demo PRIVATE glfw) # 需要可视化时打开
17
18 # ---- 把模型目录拷到可执行文件旁边，便于按相对路径加载 ----
19 add_custom_command(TARGET demo POST_BUILD
20     COMMAND ${CMAKE_COMMAND} -E copy_directory
21         "${CMAKE_CURRENT_SOURCE_DIR}/model"
22         "$<TARGET_FILE_DIR:demo>/model")
23
24 # ---- 自动拷贝 MuJoCo 动态库 ----
25 add_custom_command(TARGET demo POST_BUILD
26     COMMAND ${CMAKE_COMMAND} -E copy_if_different
27         "$<TARGET_FILE:mujoco::mujoco>"
28         "$<TARGET_FILE_DIR:demo>")

```

逐条解释：

- **CONFIG REQUIRED**: 只接受 config 模式的包 (即必须找到官方导出的 `mujocoConfig.cmake`)，找不到就立即报错而不是悄悄回退到「猜路径」的 `module` 模式。写死这一条能让问题在配置阶段暴露。
- **target_link_libraries(... PRIVATE ...)**: `mujoco::mujoco` 同时携带了头文件目录与链接信息，因此不需要手写 `include_directories` 或 `link_directories`。这正是现代 CMake 的用法。
- **\$<TARGET_FILE:mujoco::mujoco>** 在 Windows 上会展开成 `mujoco.dll` 的完整路径，**\$<TARGET_FILE_DIR:demo>** 则是输出目录。两行 `POST_BUILD` 就解决了「每次手动拷 DLL」和「模型找不到」两个问题。
- **CMAKE_CXX_STANDARD 17**: MuJoCo 的官方要求就是 C++17。

5.5 完整模板 B：手工导入库版

若 SDK 里没有 CMake 配置文件，就自己声明一个 IMPORTED 目标。把下面这段放在 find_package 的位置即可，其余部分与模板 A 完全相同：

```
1 # MuJoCo SDK 根目录，可在命令行覆盖：-DMUJOCO_DIR=...
2 set(MUJOCO_DIR "D:/mujoco/mujoco-3.14.0" CACHE PATH "MuJoCo SDK root")
3
4 if(NOT EXISTS "${MUJOCO_DIR}/include/mujoco/mujoco.h")
5     message(FATAL_ERROR "MUJOCO_DIR 指向不对：${MUJOCO_DIR}")
6 endif()
7
8 add_library(mujoco_imported SHARED IMPORTED GLOBAL)
9 set_target_properties(mujoco_imported PROPERTIES
10     IMPORTED_LOCATION      "${MUJOCO_DIR}/bin/mujoco.dll"
11     IMPORTED_IMPLIB        "${MUJOCO_DIR}/lib/mujoco.lib"
12     INTERFACE_INCLUDE_DIRECTORIES "${MUJOCO_DIR}/include"
13     INTERFACE_COMPILE_DEFINITIONS "MUJOCO_DLL")
14
15 add_library(mujoco::mujoco ALIAS mujoco_imported)
```

这样命名之后，后续的 target_link_libraries 与模板 A 写法完全一致（都用 mujoco::mujoco），以后 SDK 升级成带 CMake 包的版本，只要删掉这一段、换回 find_package 即可。

- IMPORTED_LOCATION 是运行时用的 DLL，IMPORTED_IMPLIB 是链接时用的导入库。两者都要写，而且要分清楚，这是 Windows 上独有的概念。
- GLOBAL 让该目标在子目录里也可见；多目录工程里需要。
- 开头那个 EXISTS 检查是廉价而有效的防线：路径写错时立刻报错，而不是等到链接期报一堆 LNK2019。

5.6 构建与运行

5.6.1 Visual Studio 生成器（多配置）

```
1 cmake -S . -B build -G "Visual Studio 17 2022" -A x64 ^
2     -Dmujoco_DIR="D:/mujoco/mujoco-3.14.0/lib/cmake/mujoco"
3
4 cmake --build build --config Release
5
```

```
6 .\build\Release\demo.exe
```

多配置生成器的两个坑

1. `-DCMAKE_BUILD_TYPE=Release` 对多配置生成器无效，必须在构建时用 `--config Release`。忘了写就会得到 Debug 版，仿真速度差数倍。
2. Debug 与 Release 的运行时库不同（`/MDd` 与 `/MD`）。不要把 Debug 版程序和 Release 版 DLL 混用，也不要拿 Debug 版的 `.lib` 去链接 Release 版工程，否则会出现 LNK2038 或运行期随机崩溃。

5.6.2 Ninja 生成器（单配置）

在「x64 Native Tools Command Prompt for VS 2022」中：

```
1 cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release ^
2     -Dmujoco_DIR="D:/mujoco/mujoco-3.14.0/lib/cmake/mujoco"
3 cmake --build build
4 .\build\demo.exe
```

Ninja 下 `CMAKE_BUILD_TYPE` 是有效的，且增量构建明显更快；缺点是必须在配好 `cl` 环境的窗口里运行。

5.6.3 在 Visual Studio 里打开

两种方式：直接双击 `build` 目录下的 `my_sim.sln`；或者用 VS 的「打开本地文件夹」功能选中工程根目录（VS 会直接读 `CMakeLists.txt`，无需先生成工程文件，改 `CMakeLists.txt` 会自动重新配置）。后者更适合经常改构建配置的场景。

5.7 CMakePresets.json：把命令行固化下来

每次手打一长串 `-D` 参数既繁琐又容易写错。`CMakePresets.json` 把这些参数固化进仓库，团队里每个人、以及 CI，用的都是同一套配置。放在工程根目录：

```
1 {
2   "version": 6,
3   "configurePresets": [
4     {
5       "name": "vs2022-release",
6       "displayName": "VS2022 x64 Release",
7       "generator": "Visual Studio 17 2022",
8       "architecture": "x64",
```

```

9      "binaryDir": "${sourceDir}/build/vs2022",
10     "cacheVariables": {
11         "CMAKE_PREFIX_PATH": "D:/mujoco/mujoco-3.14.0"
12     }
13 },
14 {
15     "name": "ninja-release",
16     "displayName": "Ninja Release",
17     "generator": "Ninja",
18     "binaryDir": "${sourceDir}/build/ninja",
19     "cacheVariables": {
20         "CMAKE_BUILD_TYPE": "Release",
21         "CMAKE_PREFIX_PATH": "D:/mujoco/mujoco-3.14.0"
22     }
23 },
24 ],
25 "buildPresets": [
26     { "name": "vs2022-release", "configurePreset": "vs2022-release", "
27       configuration": "Release" },
28     { "name": "ninja-release", "configurePreset": "ninja-release" }
29 ]

```

之后命令缩短为:

```

1 cmake --preset vs2022-release
2 cmake --build --preset vs2022-release

```

把路径交给环境变量 CMakePresets.json 里可以写 "\$envMUJOCO_SDK" 引用环境变量, 这样配置文件就能进入版本库而不绑定某台机器的路径: 先 `setx MUJOCO_SDK D:\mujoco\mujoco-3.14.0`, 再在 `cacheVariables` 里引用它。

5.8 路线三: vcpkg 与源码安装

5.8.1 vcpkg

vcpkg 是一个 C++ 包管理器, 装上之后 MuJoCo 与 GLFW 都能一条命令到位, 并由它生成 CMake 配置, 工程侧只需要标准的 `find_package`:

```

1 git clone https://github.com/microsoft/vcpkg

```

```

2 .\vcpkg\bootstrap-vcpkg.bat
3 .\vcpkg\vcpkg install mujoco glfw3
4
5 cmake -S . -B build -G "Visual Studio 17 2022" -A x64 ^
6     -DCMAKE_TOOLCHAIN_FILE=D:/vcpkg/scripts/buildsystems/vcpkg.cmake

```

此时 CMakeLists.txt 里写官方文档给出的两行即可：

```

1 find_package(mujoco CONFIG REQUIRED)
2 target_link_libraries(main PRIVATE mujoco::mujoco)

```

vcpkg 的两个注意点

1. **版本滞后**：vcpkg 仓库里的 MuJoCo 版本通常落后于官方最新发布（例如仓库里是 3.10.0 而官方已到 3.14.0）。需要新特性时应改用官方预编译包或源码安装。
2. **Windows 上不支持静态链接**：该 port 声明了 `supports: !(windows & static)`，因此只能用动态库（这也正是要拷 DLL 的原因）。

5.8.2 从源码安装到自己的目录

需要修改引擎、或者想要一份带完整 CMake 包配置的安装时，把 MuJoCo 编译并安装到一个自定义前缀：

```

1 git clone https://github.com/google-deepmind/mujoco.git
2 cmake -S mujoco -B mujoco/build -G "Visual Studio 17 2022" -A x64 ^
3     -DCMAKE_INSTALL_PREFIX=D:/mujoco/install ^
4     -DCMAKE_BUILD_TYPE=Release
5 cmake --build mujoco/build --config Release
6 cmake --install mujoco/build --config Release

```

安装后的目录布局是标准的 CMake 前缀：头文件在 `include/mujoco`，DLL 在 `bin`，CMake 包配置在 `lib/cmake/mujoco`。于是工程侧只要：

```

1 cmake -S . -B build -G "Visual Studio 17 2022" -A x64 -DCMAKE_PREFIX_PATH=
    D:/mujoco/install

```

就能直接用模板 A。这是三种方式里最”标准”的一条路，代价是要先花时间编译一次。

5.9 排错表

表 5.5: CMake 配置与构建常见问题

现象	原因	处理
Could not find a package configuration file provided by "mujoco"	mujoco_DIR 没给或指错	用 5.2 节的命令找到 <code>*Config.cmake</code> 所在目录；或改用模板 B
Target "mujoco::mujoco" not found	包没找到，但 <code>target_link_libraries</code> 照写了	先解决上一条；确认 <code>find_package(mujoco CONFIG REQUIRED)</code> 在 <code>add_executable</code> 之前
Target "mujoco::glfw" not found	目标名不存在	改为 <code>glfw</code> ，并单独准备 GLFW (4.4.2 节)
LNK2019 / LNK2038	架构或配置不一致	统一 x64、统一 Release；模板 B 下检查 <code>IMPORTED_IMPLIB</code>
CMake Error: could not find CMAKE_CXX_COMPILER	没装 C++ 工具链	装「使用 C++ 的桌面开发」工作负载
CMake 3.x is required	版本过低	装新版 CMake，注意 <code>where cmake</code> 的顺序
运行时找不到 <code>mujoco.dll</code>	只拷了 <code>exe</code>	用模板 A 的 <code>POST_BUILD</code> 自动拷贝
改了 <code>CMakeLists.txt</code> 但没生效	多配置生成器的缓存	删掉 <code>build</code> 目录重新配置（最省事），或 <code>cmake --fresh</code>

第六章 从源码编译 MuJoCo（可选）

第四章与第五章用的都是官方预编译库。绝大多数情况下这就够了，官方也明确建议「不打算修改引擎核心代码的用户使用预编译库」。本章说明什么情况下才值得自己编译，以及怎么做。

6.1 什么时候需要自己编译

表 6.1: 需要自行编译的场合

目的	说明
修改引擎	改动力学、接触模型、求解器，或加自己的传感器/执行器（后者更推荐用插件机制，不必改引擎源码）
换渲染后端	用 Filament 渲染器（MUJOCO_USE_FILAMENT=ON）或开启它的 mjr 兼容层
接 OpenUSD	编译带 USD 支持的目标（MUJOCO_WITH_USD=ON）
要一份完整安装	需要一个带完整 CMake 包配置的「标准前缀」安装（5.7 节），供多个工程共享
性能调优	用 Clang 自行构建，官方指出 Windows 上 Clang 的性能优于 MSVC
固定版本 / 内部魔改	需要冻结某个提交，或在公司内网维护自己的分支

不要为了「装 **Python** 绑定」而编译 `pip install mujoco` 的 wheel 已经自带引擎，不需要先编译 C++ 库。只有在你修改了 Python 绑定本身时，才需要按官方文档从源码构建 Python 包（那是一条独立且相当繁琐的流程）。

6.2 准备工作

表 6.2: 源码编译所需的工具

工具	版本	说明
Git	任意近期版本	克隆源码
CMake	3.16 及以上	这是官方源码里写明的下限；实际使用建议 3.24 以上
C++17 编译器	MSVC (VS2019+) / Clang / GCC	官方对 Windows 的要求是 VS2019 或更高，且 Windows SDK 版本不低于 10.0.22000
网络	——	构建系统会用 CMake 的 FetchContent 从上游仓库自动下载依赖 (ccd、qhull、tinyobjloader、tinyxml2、lodepng、GLFW 等)，因此首次配置必须联网
磁盘与内存	约 5 GB 空闲	依赖树与中间产物不小；并行构建时内存占用较高

先克隆，不用 `--recursive` MuJoCo 的依赖不是 git 子模块，而是配置阶段用 FetchContent 拉取的。因此 `git clone https://github.com/google-deepmind/mujoco.git` 即可，不需要（也没有）`--recursive`。若所在网络访问 GitHub 受限，可以在配置时给 Git 设置代理，或提前把依赖仓库镜像到内网并用 `FETCHCONTENT_SOURCE_DIR` 指过去。

6.3 配置、构建与安装

```
1 git clone https://github.com/google-deepmind/mujoco.git
2 cd mujoco
3
4 cmake -S . -B build -G "Visual Studio 17 2022" -A x64 ^
5     -DCMAKE_BUILD_TYPE=Release ^
6     -DCMAKE_INSTALL_PREFIX=D:/mujoco/install
7
8 cmake --build build --config Release --parallel
9
10 cmake --install build --config Release
```

四步分别对应：克隆、配置（生成工程）、构建、安装到指定前缀。安装完成后的目录符合 CMake 的标准前缀布局：

```

1 D:/mujoco/install/
2 |-- bin/                mujoco.dll
3 |-- include/mujoco/     headers
4 |-- lib/                mujoco.lib
5 |   `-- cmake/mujoco/   mujocoConfig.cmake, mujocoTargets.cmake, ...
6 `-- share/mujoco/       model collection

```

有了这个前缀，第五章的模板 A 只要把 CMAKE_PREFIX_PATH 指过去就能用：

```

1 cmake -S my_sim -B my_sim/build -G "Visual Studio 17 2022" -A x64 ^
2     -DCMAKE_PREFIX_PATH=D:/mujoco/install

```

6.4 常用 CMake 选项

表 6.3: 官方根 CMakeLists.txt 中的主要选项

选项	默认	作用
MUJOCO_BUILD_EXAMPLES	ON	构建 <code>sample</code> 目录下的示例程序
MUJOCO_BUILD_SIMULATE	ON	构建交互式查看器 <code>simulate</code>
MUJOCO_BUILD_TESTS	ON	构建单元测试；只想要库时关掉可显著缩短构建时间
MUJOCO_TEST_PYTHON_UTIL	ON	构建 Python 绑定的工具库（依赖 Python）
MUJOCO_BUILD_STUDIO	OFF	构建实验性的 MuJoCo Studio
MUJOCO_USE_FILAMENT	OFF	使用 Filament 渲染器替换经典 OpenGL 渲染器
MUJOCO_USE_FILAMENT_MJR_COMPAT	OFF	Filament 的 mjr 兼容层
MUJOCO_WITH_USD	OFF	编译 OpenUSD 相关目标
CMAKE_INSTALL_PREFIX	——	安装前缀，见上一节
CMAKE_BUILD_TYPE	——	单配置生成器下务必设为 <code>Release</code>

只想要一个干净、够用的库时，一条命令即可（关掉测试与示例）：

```

1 cmake -S . -B build -G "Visual Studio 17 2022" -A x64 ^
2     -DCMAKE_BUILD_TYPE=Release ^
3     -DMUJOCO_BUILD_TESTS=OFF ^
4     -DMUJOCO_BUILD_EXAMPLES=OFF ^
5     -DMUJOCO_TEST_PYTHON_UTIL=OFF ^
6     -DCMAKE_INSTALL_PREFIX=D:/mujoco/install

```

查看全部选项 配置一次之后，构建目录里的 `CMakeCache.txt` 记录了所有选项；也可以用 `cmake -LAH -S . -B build` 让 CMake 打印带说明的选项列表。

6.5 验证构建结果

1. 到 `build/bin/Release`（或安装前缀的 `bin`）下运行 `simulate.exe`，打开自带模型验证渲染；
2. 用第五章的模板 A 编译一个最小程序，确认 `find_package(mujoco)` 能找到你刚安装的前缀；
3. 在程序里检查版本一致性：`mjVERSION_HEADER` 与 `mj_version()` 应当相等。

6.6 常见编译失败

表 6.4: 源码构建常见问题

现象	原因	处理
配置阶段卡在下载依赖	网络无法访问 GitHub	配置 Git 代理；或把依赖镜像到内网后用 <code>FETCHCONTENT_SOURCE_DIR</code> 指定本地副本
CMake 3.16 or higher is required	CMake 版本过低	安装新版 CMake
MSVC 报 Windows SDK 相关错误	SDK 版本低于 10.0.22000	在 VS 安装器里补装新版 Windows SDK
cl 找不到	没在开发者命令行里运行	见 4.2.2 节
编译到一半内存耗尽	并行度过高、或同时构建测试与 Filament	降低并行度 (<code>--parallel 4</code>)，关掉 <code>MUJOCO_BUILD_TESTS</code>
运行时报非法指令	CPU 不支持 AVX	换机器；官方预编译库与默认构建都要求 AVX
Filament 构建失败	额外依赖很重、平台差异多	除非确实需要，否则保持 <code>MUJOCO_USE_FILAMENT=OFF</code>

建议 先用官方预编译库把项目跑通，确认是「模型/代码的问题」而不是「构建的问题」，再考虑自行编译。自行编译带来的收益主要体现在改引擎与性能调优上，而这部分工作只占实际项目的一小部分时间。

第七章 SolidWorks 建模与模型导出

前六章用的是官方自带模型。真正的研究与项目里，模型往往来自 CAD。本章解决「怎么从 SolidWorks 里把 MuJoCo 要的东西拿出来」，第八章解决「拿出来之后怎么写进 MJCF」。

7.1 先明确 MuJoCo 要什么

一次成功的导出，本质是交付下面四样东西：

1. **一套网格**：每个连杆一个（或一组）三角网格，格式为 OBJ 或二进制 STL（MuJoCo 还支持自家的二进制 MSH，但旧格式已废弃，不建议使用）；
2. **一棵运动学树**：谁是谁的父连杆、关节装在哪里、绕哪个轴转、限位多少；
3. **一套惯性参数**：质量、质心、惯性张量；
4. **一个统一的空间约定**：长度单位为米、 z 轴向上、右手系。

定义 7.1（MuJoCo 的空间与角度约定） MuJoCo 的默认世界朝向是 z 轴向上，坐标系为右手系，姿态用四元数表示且分量顺序为 $wxyz$ （单位四元数为 "1 0 0 0"）。MJCF 中角度默认按度解释（`compiler` 的 `angle` 属性），而 URDF 一律按弧度解释；但编译进 `mjModel` 之后，所有角度量都是弧度。

注：SolidWorks 与 URDF 同样使用 z 轴向上的右手系，因此通常不需要做 y 轴向上到 z 轴向上的旋转。若模型导入后整体「躺倒」，先检查是不是自己在导出时选了一个额外的输出坐标系，而不是怀疑引擎做了自动旋转。

7.2 SolidWorks 侧的六个建模约定

这六条决定了后续工作量的大小。装配体越「干净」，MJCF 越短。

7.2.1 单位：毫米建模，米导出

工程上习惯用毫米建模，而 MuJoCo 需要一个自洽的单位制，实践中统一用米-千克-秒。两种做法：

- **推荐：** SolidWorks 里仍用毫米（图纸、加工都方便），导出网格后在 MJCF 里用 `scale` 把网格缩放 0.001 倍，其余数值（关节位置、限位、质量）直接按米填写；
- 也可以把文档单位直接改成米（工具 → 选项 → 文档属性 → 单位），导出即为米，但后续与图纸、加工数据交接时容易出错。

```
1 <asset>
2   <mesh name="base_link" file="base_link.STL" scale="0.001 0.001 0.001"/>
3 </asset>
```

最常见的一个错误 忘记写 `scale`，结果机器人被放大 1000 倍：看不见、一放手就飞出去、接触力大到发散。判断口诀：如果模型数值上「大得离谱」，先查单位。

7.2.2 坐标系：原点放在关节轴线上

每个连杆都需要一个参考坐标系，它决定了：

- 网格自身坐标系的原点与朝向（由导出时选择的输出坐标系决定）；
- 关节的位置（MJCF 里 `body` 的 `pos` 或关节所在的 `body` 原点）。

实践建议：在每个连杆与父连杆的连接处（关节中心）建一个参考坐标系，命名为 `CS_<link>`，导出时以它为输出坐标系。这样每个连杆的网格原点就落在关节中心上，MJCF 里写关节位置时只需写父连杆末端到子连杆关节中心的偏移，几何直观、不易出错。

装配体 vs 零件 从零件导出时，「输出坐标系」通常只能选零件的原点或用户坐标系；从装配体导出时可以选择任意参考坐标系。如果零件自身的原点不在关节轴线上，最省事的办法是在装配体里导出，或者接受偏移、在 MJCF 里用引用网格的 `geom` 的 `pos` 做补偿。

7.2.3 拆分：一个连杆一个子装配体

MuJoCo 的物体（`body`）是运动学树上的节点，网格只是挂在它上面的几何体。因此要按运动学意义上的连杆来组织装配体：

- 同一连杆的所有零件（本体、法兰、螺栓、电机外壳）放进一个子装配体；
- 相邻连杆之间不要有共享零件，否则无法拆分；

- 紧固件、垫片、铭牌等对动力学没有影响的零件**直接压缩掉**（用显示状态隐藏，见 7.3.6 节）。

7.2.4 命名：让 XML 里的名字和 CAD 一致

命名混乱是后期调试最大的时间黑洞。建议：

- 连杆名用 `base_link`、`link1`、`link2`……或语义名 `upper_arm`、`forearm`；
- 关节名与连杆对应：`joint1`、`joint2`；
- 参考坐标系加前缀：`CS_link1`；
- 网格文件名与连杆名一致：`link1.STL`。

这样在 MJCF、Python、C++ 里引用同一件事时用的是同一个字符串，模型改动时改动量最小。

7.2.5 质量属性：材质密度必须真实

MuJoCo 可以由几何体**反推**质量与惯量（用密度乘以体积），也可以直接给 `inertial`。无论哪种，前提是 CAD 里的质量属性是可信的：

- 给每个零件指定**真实材质**（密度），不要用默认的「普通碳钢」；
- 不要用「覆盖质量属性」手改质量而不改惯量；
- 导出的 URDF 里 `inertial` 一旦缺失或非正定，MuJoCo 编译时会报错或给出不合理结果。

7.2.6 碰撞体：单独做简化件

MuJoCo 的碰撞检测**只使用网格的凸包**。直接拿外形复杂的精模当碰撞体，会同时带来三个问题：慢、结果失真、非凸部分「填实」。正确做法是：

- 为每个连杆单独做一个**包络件**（skin part），用圆柱、长方体等简单形状把真实外形包住；
- 包络件材质设为「空气」或质量置零，并对质量属性计算做排除（`Exclude from BOM` 与质量属性里的「不包括」），使它不影响惯性；
- 精模只用于显示（visual），包络件只用于碰撞（collision）。

为什么值得多花这一步 一个几十万面的网格当碰撞体，凸包计算和接触求解都会明显变慢；而包含大量细节的凸包还会让相邻连杆「粘」在一起。官方在整理 SW2URDF 大装配体的教程里给的建议与此一致：用包络件做粗碰撞几何，用 MeshLab 之类的工具减面。

7.3 方案 A：用 SW2URDF 插件导出 URDF

这条路线能**保留运动学树**：插件让你在 SolidWorks 里逐个连杆指定组成零件、命名关节、选择关节类型与参考轴，最后导出一个标准 URDF 包。之后 MuJoCo 可以直接加载 .urdf 并另存为 MJCF（8.4 节）。

7.3.1 获取与安装

- 官方名称是 **SolidWorks to URDF Exporter**，托管在 GitHub 的 ros 组织下（仓库名 `solidworks_urdf_exporter`），安装后在插件列表里显示为 **SW2URDF**；
- 在 GitHub 的 Releases 页面下载安装包，或用 Visual Studio 自行编译其中的 C# 源码；
- 环境要求：**.NET Framework 4.7.2 及以上**；仅支持 **64 位** SolidWorks；安装程序会往 SolidWorks 安装目录写文件，需要管理员权限；
- 已知缺陷：SolidWorks 2018 **Service Pack 4 及更早**存在 STL 导出 bug，会让该插件无法使用，请升级到 SP5 或改用 2019 及更高版本。

安装后在 工具 → 插件里能看到 SW2URDF，勾选后即可使用。

7.3.2 导出步骤

1. 打开装配体，把各个关节摆到你希望作为零位的姿态（导出的 URDF 会以当前姿态为基准）；
2. 选择 工具 → *Export to URDF*（1.5.1 及更早的版本在 文件 → *Export to URDF*），界面左侧出现插件的属性管理器；
3. 先指定**基座连杆**（base link）：给它起名、勾选属于它的零件或子装配体。基座没有关节字段；
4. 逐个添加连杆：每个连杆都要有唯一的**连杆名**与**关节名**，勾选属于它的组件，选择**关节类型**（旋转 / 移动 / 固定等）；
5. 需要精确控制关节位置与轴向时，先在 SolidWorks 里建好**参考坐标系**和**参考轴**，再在插件界面里从下拉列表中选择它们；
6. 用右键菜单与拖拽维护连杆树，直到与真实机器人的拓扑一致；
7. 点击 *Preview and Export...*，选择输出目录，插件会生成 URDF 包。

三个容易卡住的限制

1. 只能在设计树里选组件：在图形区点选通常只能选到零件，选不到子装配体。
2. 只能选到两层深度：出于性能考虑，插件只允许选择装配体中两层以内的零件或子装配体。更深的层级要先在 SolidWorks 里提升层次。
3. 关节属性页的修改不会保存：在这一页里改的关节原点、轴向等信息不会被记住，下次还要重填。若对自动生成的参考坐标系/轴不满意，应在该页点取消（此时连杆名会保存），回到 SolidWorks 修改参考几何，再重新执行一次导出。

7.3.3 导出产物

插件写出的是一个 ROS 风格的包：urdf 文件、package.xml 清单，以及网格与纹理目录。内容大致如下（目录名以实际导出为准）：

```
1 robot_description/  
2 |-- package.xml  
3 |-- urdf/  
4 |   |-- robot.urdf  
5 |-- meshes/  
6 |   |-- base_link.STL  
7 |   |-- link1.STL  
8 |   |-- ...  
9 |-- textures/
```

路径必须改 URDF 里的网格路径是 ROS 的 package:// 形式，MuJoCo 不认识这种路径。两种处理办法：把网格文件与 URDF 放到同一目录树里并改写为相对路径（推荐，配合 meshdir）；或者把模型整体存成 .mjz 归档，让网格跟着 XML 一起走。

7.3.4 大装配体的整理技巧

官方在「为大装配体整理 SolidWorks 模型」的教程里给了几条很实用的建议：

- 用显示状态（Display States）隐藏紧固件，不要用配置（Configuration）。理由是插件计算质量属性时会忽略被压缩的组件，而配置恰恰通过「压缩组件或特征」实现隐藏，容易连累配合关系；
- 为每个连杆建一个包络件做碰撞几何，材质设为「空气」；

- **网格减面**：SolidWorks 最粗的 STL 细分精度对仿真来说仍然太细，用 MeshLab 之类的工具做减面；
- **视觉网格与碰撞网格要跑两遍导出**（一次导出精模、一次导出包络件），然后手工把两组 <collision> 合并到同一个连杆里。

7.4 方案 B：直接导出 OBJ / STL

如果关节树你打算自己写（或者模型很简单），可以只要网格：在 SolidWorks 里用 文件 → 另存为 选择 OBJ 或 STL，并在对话框的 选项里 设置导出参数。

对话框字段随后续版本变化 SolidWorks 各版本的 OBJ/STL 导出对话框字段并不完全一致，官方帮助文档也未公开可检索。因此请以你本机对话框上的实际名称为准，逐项确认下面这几件事：

1. **单位**：导出的是毫米还是米（决定要不要写 `scale="0.001 ..."`）；
2. **输出坐标系**：以零件原点、装配体原点，还是某个参考坐标系为基准；
3. **精度 / 分辨率**：三角面越多越精细，也越慢；碰撞体不要用最高精度；
4. **装配体是否输出为单一文件**：应选择按零件分别输出，否则整个装配体会被合并成一个网格，无法按连杆使用；
5. **是否三角化**：MuJoCo 只接受三角网格。

两种格式在 MuJoCo 侧的区别见表 7.1。简单结论是：**只做显示和碰撞时两者都行；需要贴纹理时必须用 OBJ**（因为只有 OBJ/MSH 能携带纹理坐标）。

表 7.1: OBJ 与 STL 在 MuJoCo 中的差别

格式	纹理坐标	法线	备注
OBJ	支持	支持	可以携带 UV；但 MuJoCo 不读 MTL 材质文件 ，颜色与纹理要在 XML 里用 <code>material</code> 重新指定
STL	不支持	不支持	二进制 STL 会做重复顶点合并；STL 无法携带纹理坐标，贴图无从谈起

7.5 导出之后：先量一量

不管走哪条路线，拿到文件后**先做一次量化检查**，比在 MJCF 里盲猜要快得多。用 Python 读一次网格的包围盒与顶点数：

```

1 import numpy as np
2
3 def read_stl_bounds(path):
4     with open(path, "rb") as f:
5         f.seek(80)
6         n = int.from_bytes(f.read(4), "little")
7         data = np.frombuffer(f.read(n * 50), dtype=np.uint8).reshape(n,
8                               50)
9         tri = data[:, 12:48].copy().view("<f4").reshape(n * 3, 3)
10        print("triangles:", n)
11        print("min:", tri.min(axis=0))
12        print("max:", tri.max(axis=0))
13        print("size:", tri.max(axis=0) - tri.min(axis=0))
14
15 read_stl_bounds("meshes/link1.STL")

```

- **size** 应当在米的量级（例如 0.30 表示 30 cm）。若是 300 这样的数，说明导出的是毫米，MJCF 里必须写 `scale="0.001 0.001 0.001"`；
- **min** 与 **max** 告诉你要在 MJCF 里把网格平移多少——但如果导出时把坐标系选在了关节中心上，这一步就可以省略；
- **triangles** 过大（例如超过 10 万）时应减面，尤其是碰撞网格。

7.6 两条路线对照与常见坑

表 7.2: 两条导出路线对照

对比项	方案 A: SW2URDF	方案 B: 直接导出网格
产出	URDF + 网格 + 清单	只有网格文件
运动学树	插件里可视化搭建，可保留	全部手写
适合场合	多自由度机器人、关节多、结构复杂	简单机构、单连杆、只要几何
上手成本	需要理解插件界面与参考几何	立刻可用
后续维护	改结构后在 SolidWorks 里改一次、重新导出	改结构要同时改网格与 XML
与 MJCF 的衔接	用 MuJoCo 直接编译 URDF，再另存为 MJCF（8.4 节）	直接写 MJCF，用 <code><mesh></code> 引入网格

表 7.3: CAD 导出阶段的常见坑

现象	原因	解决
机器人被放大 1000 倍 整个机器人是一个刚体	网格单位是毫米而没写 <code>scale</code> 装配体被导出成一个合并网格	加 <code>scale="0.001 0.001 0.001"</code> 按零件/子装配体分别导出，或改用 SW2URDF
模型「躺倒」	导出时选了带旋转的输出坐标系	检查输出坐标系，重新导出一份纯平移的
贴图不显示	用了 STL，或指望 MTL 生效	改用 OBJ 并携带 UV；在 XML 里用 <code><texture></code> 与 <code><material></code> 指定
加载时报「翻转的面」	部分三角面的法线朝内	在 CAD 或 MeshLab 里统一法线朝外；MuJoCo 会对 OBJ/XML 网格检查翻转面并报错
阴影出现条纹	存在重复面（同一面正反各一份）	在 MeshLab 里删除重复面
编译报错、指向某个网格	存在面积过小的退化三角形	删除退化面后重新导出
仿真中相邻连杆互相「粘住」	碰撞体用了精模，凸包把间隙填实	用包络件做碰撞体，或把碰撞体放进不参与碰撞的组

下一步 文件已经有了，第八章把它们组织成一个能编译、能跑的 MJCF 模型：目录怎么摆、`<mesh>` 怎么写、材质与纹理怎么设、URDF 怎么转成 MJCF。

第八章 网格处理与 MJCF 模型编写

第七章解决「把网格拿出来」,本章解决「把它组织成一个能编译、能跑模型」。MuJoCo 的模型描述语言叫 **MJCF** (*MuJoCo XML Format*), 它比 URDF 表达能力强得多: 闭环约束、肌腱、柔体、执行器与传感器的内部动力学都能直接写。好消息是大多数属性都有合理的默认值, 一个能跑的模型往往只有几十行。

8.1 工程目录组织

8.1.1 推荐布局

官方模型库 MuJoCo Menagerie 的做法值得直接照搬: 机器人本体一个 XML, 网格放在 `assets`, 再用一个 `scene.xml` 把它装进场景。

```
1 two_link_arm/  
2 |-- arm.xml           robot only: bodies, joints, geoms, actuators  
3 |-- scene.xml         include arm.xml + floor + lights + textures  
4 `-- assets/  
5     |-- link1.STL  
6     |-- link2.STL  
7     `-- base.STL
```

`arm.xml` 开头用 `meshdir` 指明网格目录:

```
1 <mujoco model="two_link_arm">  
2   <compiler angle="radian" meshdir="assets"/>  
3   ...
```

`scene.xml` 则只是一个小外壳, 用 `<include>` 把本体装进来, 再加上地面、光照与天空盒:

```
1 <mujoco model="two_link_arm scene">  
2   <include file="arm.xml"/>  
3   <visual>  
4     <headlight diffuse="0.6 0.6 0.6" ambient="0.3 0.3 0.3" specular="0 0 0"  
       "/>
```

```

5     <global azimuth="120" elevation="-20"/>
6 </visual>
7 <asset>
8     <texture type="skybox" builtin="gradient"
9         rgb1="0.3 0.5 0.7" rgb2="0 0 0" width="512" height="3072"/>
10    <texture type="2d" name="groundplane" builtin="checker" mark="edge"
11        rgb1="0.2 0.3 0.4" rgb2="0.1 0.2 0.3"
12        markrgb="0.8 0.8 0.8" width="300" height="300"/>
13    <material name="groundplane" texture="groundplane"
14        texuniform="true" texrepeat="5 5" reflectance="0.2"/>
15 </asset>
16 <worldbody>
17     <light pos="0 0 1.5" dir="0 0 -1" directional="true"/>
18     <geom name="floor" size="0 0 0.05" type="plane" material="groundplane"
19     />
20 </worldbody>
21 </mujoco>

```

这样拆分的好处是：同一个机器人可以配多个场景（空场景、带障碍物的场景、带相机的场景），而本体只维护一份。

8.1.2 资源搜索规则

网格与纹理找不到，是 MJCF 最常见的报错来源。MuJoCo 的搜索顺序是确定的：

1. file 里写的是**绝对路径** → 直接使用，不再做任何拼接；
2. meshdir（或 texturedir）已设置且**本身是绝对路径** → 用它拼接文件名；
3. 否则 → **主 MJCF 文件所在目录**，拼接 meshdir（若设置了），再拼接文件名。

其中 assetdir 会同时设置 meshdir 与 texturedir，而这两个属性各自的取值优先于 assetdir。另外：

- <include file="..."> 的路径相对于主 MJCF 文件所在目录；
- compiler 的 strippath="true" 会丢弃文件名里的路径信息，用于「换机器后路径全变了」的场合；
- 模型存成 .mjz（zip 归档）时，网格与纹理随包携带，彻底避开路径问题——这也是交付模型时最稳的做法。

三个属性，一条口诀 meshdir 管网格、texturedir 管纹理、assetdir 一管两个。不确定时先写 <compiler assetdir="assets"/>，再把网格放进去。

8.2 网格与材质的写法

8.2.1 网格元素

表 8.1: <mesh> 的常用属性

属性	默认值	说明
file	——	网格文件，扩展名必须是 obj、stl 或 msh（不区分大小写），格式由扩展名决定
name	文件名	省略时取「去掉路径与扩展名的文件名」
scale	1 1 1	三个方向分别缩放；毫米转米就靠它
smoothnormal	false	为真时对顶点法线做平滑，否则保留面的硬边
maxhullvert	-1	碰撞凸包的顶点数上限（-1 表示不限）
inertia	legacy	由网格推算惯量的方式，推荐 convex；exact 要求网格水密，否则直接报错
material	——	网格未指定材质时的兜底材质
content_type	由扩展名推断	显式指定媒体类型，一般不用写

<mesh> 没有 pos 与 quat 把网格摆到正确位置，用的是引用它的 geom 的 pos/quat，而不是 <mesh> 自己的属性。初学时容易在 <mesh> 上找一个 pos 属性而找不到，原因就在这里。

8.2.2 外观：材质与纹理

MuJoCo 的网格本身没有颜色，颜色来自引用它的几何体所指定的材质。这一点与常见三维软件的习惯不同，务必注意：

- OBJ 附带的 MTL 材质文件不会被读取；
- <texture> 只支持 PNG 与 KTX 两种图像格式，或使用 builtin 生成程序化纹理（棋盘、渐变、天空盒等）；
- 纹理坐标只能来自 OBJ、MSH 或在 XML 里显式给出的 texcoord，STL 无法携带，且这几种方式不能混用；
- 一次只能给一个网格指定一种材质，复合材质（一个 OBJ 里有多个 o/g 组或多个材质）需要先拆分——这正是 8.5 节 obj2mjcf 存在的原因。

```
1 <asset>
2   <texture type="2d" name="metal_tex" file="metal.png"/>
3   <material name="metal" texture="metal_tex" texrepeat="2 2" reflectance="
    0.3"/>
4   <material name="plastic" rgba="0.9 0.4 0.1 1"/>
```

```

5   <mesh name="link1" file="link1.STL" scale="0.001 0.001 0.001" inertia="
      convex"/>
6 </asset>
7
8 <worldbody>
9   <body name="link1" pos="0 0 0.1">
10     <joint name="joint1" axis="0 0 1"/>
11     <geom name="link1_v" type="mesh" mesh="link1" material="metal"/>
12   </body>
13 </worldbody>

```

8.3 最小可运行模型

先用基本几何体（不需要任何网格文件）把结构跑通，再换成网格。下面这个二连杆机械臂可以直接复制运行，后面的章节都以它为例。

```

1 <mujoco model="two_link_arm">
2   <compiler angle="radian"/>
3
4   <default>
5     <geom type="capsule" rgba="0.7 0.7 0.75 1"/>
6     <joint type="hinge" damping="0.1" armature="0.01"/>
7   </default>
8
9   <asset>
10     <material name="steel" rgba="0.65 0.65 0.70 1"/>
11     <material name="orange" rgba="1.00 0.50 0.05 1"/>
12   </asset>
13
14   <worldbody>
15     <light pos="0 0 2" dir="0 0 -1" directional="true"/>
16     <geom name="floor" type="plane" size="2 2 0.05" rgba="0.3 0.35 0.4 1"/>
17     <
18     <body name="base" pos="0 0 0.1">
19       <geom name="base_geom" type="cylinder" size="0.06 0.05" material="
      steel"/>
20
21     <body name="link1" pos="0 0 0.05">
22       <joint name="joint1" axis="0 0 1" range="-3.1416 3.1416"/>

```

```

23     <geom name="link1_geom" fromto="0 0 0 0 0 0.3" size="0.025"
    material="orange"/>
24
25     <body name="link2" pos="0 0 0.3">
26         <joint name="joint2" axis="0 1 0" range="-2.6 2.6"/>
27         <geom name="link2_geom" fromto="0 0 0 0.25 0" size="0.02"
    material="steel"/>
28         <site name="tool" pos="0.25 0 0" size="0.01"/>
29     </body>
30 </body>
31 </body>
32 </worldbody>
33
34 <actuator>
35     <position name="act1" joint="joint1" kp="100" kv="5"/>
36     <position name="act2" joint="joint2" kp="50" kv="2"/>
37 </actuator>
38
39 <sensor>
40     <jointpos name="j1_pos" joint="joint1"/>
41     <jointpos name="j2_pos" joint="joint2"/>
42     <jointvel name="j1_vel" joint="joint1"/>
43 </sensor>
44
45 <keyframe>
46     <key name="home" qpos="0 0"/>
47 </keyframe>
48 </mujoco>

```

8.3.1 逐段解读

compiler `angle="radian"` 让所有角度按弧度解释。MJCF 的默认是度，而 URDF 与编程接口一律用弧度，显式写成弧度可以避免「90 与 1.57 搞混」这类错误。

default 「默认值机制」是 MJCF 最省事的设计：<default> 里设定的属性会自动应用到其后的所有同类元素，作用类似 HTML 中内联的 CSS。这里统一了几何体类型、颜色与关节阻尼，后面就不必重复写。

asset 只放被引用的资源：材质、纹理、网格。材质必须要有名字，因为只能按名字引用。

worldbody 运动学树的根。世界本身是第 0 号物体；地面用 **plane** 挂在世界上。

body 树上的节点。**pos** 是相对父物体坐标系的位置。注意关节与几何体都写在所属 **body** 内部，它们默认位于该 **body** 的原点。

joint 关节增加自由度。**range** 是位置限位（这里是弧度），**damping** 是黏性阻尼，**armature** 是等效到关节上的转动惯量——后两者在数值稳定性上很有用（见第九章）。

geom 几何体只有两种用途：外观与碰撞，或者用 **contype/conaffinity** 关掉碰撞而只做外观。**fromto** 让胶囊体由两个端点定义，比先摆位置再定朝向直观得多。

site「轻量几何体」：有位置与朝向，但不参与碰撞、不贡献质量，专门用来标记参考点（工具中心点、传感器安装点、肌腱路径点）。这里用它标出末端执行器位置。

actuator 执行器。**position** 是内置的位置伺服（就是 PD），**kp/kv** 分别是比例与微分增益。用它比自己在外层写 PD 更省事，也让模型自带控制律。

sensor 传感器。读数按声明顺序排列在 **data.sensordata** 中，可以用命名访问取用。

keyframe 关键帧，保存一组完整的初始状态（位置、速度、控制、时间等）。**qpos="0 0"** 表示两个关节角都为 0，配合 **mj_resetDataKeyframe** 可以一键回到初始姿态。

8.3.2 验证

```
1 import mujoco
2
3 m = mujoco.MjModel.from_xml_path("scene.xml")
4 d = mujoco.MjData(m)
5
6 print("nq =", m.nq, "nv =", m.nv, "nu =", m.nu, "nsensordata =", m.
      nsensordata)
7 print("body masses =", m.body_mass)
8
9 mujoco.mj_resetDataKeyframe(m, d, 0)
10 for _ in range(1000):
11     d.ctrl[:] = [0.5, -0.8]
12     mujoco.mj_step(m, d)
13
14 print("joint1 =", d.joint("joint1").qpos[0])
15 print("joint2 =", d.joint("joint2").qpos[0])
16 print("tool   =", d.site("tool").xpos)
```

body_mass 是校验模型是否「装对」的第一站：数值量级不对（例如抓手比整条手臂还重）说明密度或单位出了问题。

8.4 从 URDF 转成 MJCF

8.4.1 直接加载

MuJoCo 的解析器原生支持 URDF，因此第七章导出的 `robot.urdf` 可以直接打开：

```
1 import mujoco
2
3 m = mujoco.MjModel.from_xml_path("robot.urdf") # 直接编译 URDF
4 mujoco.mj_saveLastXML("robot.xml")           # 存成 MJCF
```

存成 MJCF 之后再改，比每次改 URDF 更方便——因为 MJCF 能表达 URDF 表达不了的东西。

8.4.2 转换时的默认值变化

导入 URDF 时，MuJoCo 会调整若干编译器默认值，这些差异解释了「为什么转过来以后模型看起来不一样」：

表 8.2: URDF 导入时的默认值差异

属性	URDF 下的默认	影响与处理
<code>discardvisual</code>	<code>true</code>	纯视觉的几何体（ <code>contype=conaffinity=0</code> ）、材质与纹理会被丢弃。想要保留外观，在 URDF 里加 <code><mujoco><compiler discardvisual="false"/></mujoco></code>
<code>fusestatic</code>	<code>true</code>	与父级之间没有自由度的「静态物体」会被合并进父级，这是针对 URDF 里大量无用的 dummy link 的优化；需要保留结构时关掉它
<code>angle</code>	强制为弧度	URDF 一律弧度，无需处理
<code>balanceinertia</code>	<code>false</code>	部分 URDF 的惯量在数值上不满足三角不等式，MuJoCo 会拒绝编译；置 <code>true</code> 让编译器自行修补

```
1 <robot name="robot">
2   <mujoco>
3     <compiler meshdir="../meshes" discardvisual="false" balanceinertia="
4       true"/>
5   </mujoco>
6   ...
7 </robot>
```

注：URDF 扩展标签 `<mujoco>` 是官方支持的写法，主要用途之一就是指定网格目录——因为 URDF 里的 `package://` 路径 MuJoCo 不认识。

URDF 不做模式校验 MuJoCo 解析 URDF 时不会校验属性名是否合法：拼错的属性会被静默忽略，结果就是「明明写了却没生效」。转换后发现某个设置没起作用时，先怀疑拼写，再怀疑语义。

8.4.3 转换后必须手工补的东西

URDF 只能表达 MuJoCo 能力的一个子集。转成 MJCF 之后，通常还需要手工加上：

- **执行器**：URDF 只描述「关节能出多大力」，不描述控制方式；要做仿真必须自己写 `<actuator>`；
- **传感器**：同上，URDF 里没有传感器概念；
- **接触参数**：摩擦系数、`solref/solimp` 等；
- **关键帧**：一个合理的初始姿态；
- **场景**：地面、光照、相机。

推荐的流程是：在 URDF 里只保留几何与运动学，其余全部在 MJCF 里补，并用 `<include>` 把公共部分拆出去复用。

8.5 用 obj2mjcf 处理复合网格

SolidWorks 导出的装配体网格、或从网上下载的 OBJ，常见两个问题：一个文件里有多个材质分组、纹理是 JPG。obj2mjcf 就是为此写的工具，MuJoCo Menagerie 的模型网格也是用它处理的。

```
1 pip install --upgrade obj2mjcf
2 obj2mjcf --help
```

它做的事很明确：

1. 遍历目录里的每个 *.obj，按 MTL 中引用的材质把网格拆成若干子网格，输出到以文件名为名的子目录；
2. 把纹理复制到该目录，并把 .jpg/.jpeg 转成 PNG（因为 MuJoCo 只支持 PNG）；
3. 生成一个预填好 materials、meshes、geom 的 MJCF，可以直接当作建模起点；
4. 加上 --decompose 时，调用 CoACD 做凸分解，生成 *_collision_*.obj 用于碰撞。

常用参数（名称取自其源码，实际拼写以 --help 为准）：--obj-dir、--obj-filter（正则筛选）、--save-mjcf（保存示例 XML）、--compile-model（试编译以检查错误）、--decompose、--texture-resize-percent、--overwrite、--add-free-joint。

典型用法 先把 CAD 导出的 OBJ 放进 assets/, 然后 `obj2mjcf --obj-dir assets --save-mjcf --compile-model`, 得到一个已经能编译的骨架, 再往里面加关节、执行器与传感器。

8.6 碰撞几何

8.6.1 碰撞只用凸包

官方文档的原话是: 任何三角形集合都可以作为网格加载并渲染, 但碰撞检测使用的是网格的凸包。由此推出两条实践规则:

1. **连杆外形非凸时** (例如带凹槽的机械臂), 直接用外形网格当碰撞体会让凹处被「填实」, 相邻连杆容易互相卡住; 正确做法是把非凸外形分解成若干个凸体, 都挂在同一个 body 上;
2. **网格越简单越好**: 碰撞网格与显示网格分开, 碰撞用包围盒/圆柱/胶囊或减面后的粗网格, 显示才用精模。

`maxhullvert` 用于限制凸包的顶点数上限, 模型很大时可以用它控制编译时间与内存; `inertia="convex"` 对非凸网格也能给出合理的惯量, 是推荐设置。

8.6.2 视觉与碰撞分离的写法

`contype` 与 `conaffinity` 是两个「碰撞位掩码」: 只有当两个几何体的位掩码按位与的结果非零时才会发生碰撞。把两个都设为 0 就是纯视觉几何体; 反之, 碰撞几何体可以放进默认不显示的渲染组 (group), 做到「看不见但确实在碰撞」。

```
1 <body name="link1" pos="0 0 0.05">
2   <joint name="joint1" axis="0 0 1"/>
3
4   <!-- visual: precise mesh, no collision -->
5   <geom name="link1_visual" type="mesh" mesh="link1"
6     material="orange" contype="0" conaffinity="0" group="1"/>
7
8   <!-- collision: coarse capsule, hidden by default -->
9   <geom name="link1_collision" type="capsule"
10     fromto="0 0 0 0 0 0.3" size="0.03"
11     group="3" rgba="1 0 0 0.3"/>
12 </body>
```

- `contype=conaffinity=0` 的几何体在导入 URDF 时会被 `discardvisual` 丢掉——所以想让外观保留, 一定要把该选项设为 `false`;

- 碰撞几何体默认仍然参与质量与惯量的计算，若它只是包络件，应显式给出质量或密度，或者用 `mass="0"` 排除；
- 调试时把碰撞几何体打开显示（`simulate` 里切换渲染组），能一眼看出「卡住」的原因。

8.7 常见 MJCF 错误

表 8.3: MJCF 编译期常见错误

报错或现象	原因	处理
could not open file	<code>meshdir</code> 不对，或工作目录不是 XML 所在目录	按 8.1.2 节的搜索顺序逐一排查；或改用绝对路径/存成 <code>.mjz</code>
Error: repeated name	同一类元素重名	名字只需在同类元素内唯一；检查是否有两个 <code><geom name="..."></code> 同名
Error: unknown attribute	属性名拼写错误	对照官方 XML Reference 的对应元素
翻转面报错	OBJ/XML 网格里有法线朝内的面	在 CAD 或 MeshLab 里统一法线
退化三角形报错	存在面积小于 10^{-14} 的三角形	删除退化面
模型位置不对	把 <code>pos</code> 写在了 <code><mesh></code> 上（不存在的属性）	改写在引用它的 <code>geom</code> 上
一启动就抖动	初始姿态互相穿插、时间步过大	见第九章

下一步 模型能编译、能动了，但「动得对不对」是另一回事。第九章讲怎么量化地检查模型、怎么调参数，以及一张按报错信息索引的排错表。

第九章 模型验证、调参与排错

模型能编译、能动，不等于「动得对」。本章把「怎么判断模型是对的」变成几个可以照做的检查，再给出调参与排错的速查表。

9.1 先用眼睛看一遍

拿到任何新模型，第一件事是用查看器打开，按下面的顺序看：

1. 打开模型看比例：`simulate.exe model\scene.xml`，或 Python 查看器 `python -m mujoco.viewer` 加参数 `--mjcf=scene.xml`；
2. **看比例**：机器人是不是约等于真人/真实机构的大小？如果大得看不见或小得看不清，几乎一定是单位问题（第七章）；
3. **看姿态**：连杆朝向是否符合预期，有没有整体旋转 90° （输出坐标系选错）、有没有零件跑到别的连杆上（拆分错误）；
4. **看接触体**：在右侧渲染面板里把碰撞几何所在的组打开，检查相邻连杆的碰撞体是否互相穿插；
5. **看惯性盒**：打开「等效惯性盒」（equivalent inertia box）显示，检查质量分布是否合理——一个明显偏在一边的惯性盒通常意味着某个零件的密度没设；
6. **看接触力**：拖动施加外力，观察接触点与接触力方向是否正常。

鼠标操作为：左键拖拽旋转、右键拖拽平移、滚轮缩放、按住 `Ctrl` 拖动施加外力。查看器支持拖动 MJCF 文件到窗口中打开，`Ctrl+L` 可以重新加载当前文件——改完 XML 不必重启程序。想直接得到模型全文，可用 `Ctrl+M` 把模型导出为文本文件，对比「你写的」与「编译器理解的」之间的差异。

9.2 量化自检

9.2.1 第一条：质量与惯量

```
1 import mujoco
2 import numpy as np
```

```

3
4 m = mujoco.MjModel.from_xml_path("scene.xml")
5
6 for i in range(m.nbody):
7     name = mujoco.mj_id2name(m, mujoco.mjtObj.mjOBJ_BODY, i) or f"body{i}"
8     print(f"{name:16s} mass = {m.body_mass[i]:8.4f}  "
9           f"inertia(diag) = {m.body_inertia[i]}")

```

要检查的三件事：

- **数量级：**每个连杆的质量应当与真实零件相符（例如 0.5 ~ 3 kg）；出现 10^{-9} 或 10^6 这样的数就是单位或密度问题；
- **零质量：**移动物体的质量不能为 0，引擎会直接报错；
- **正定：**惯性张量的三个主值都必须为正且满足三角不等式，否则会报警告 `mjWARN_INERTIA`（URDF 导入时可用 `balanceinertia="true"` 让编译器修补）。

9.2.2 第二条：警告计数

引擎不会抛异常，而是把异常情况记在 `data.warning` 里。每次仿真结束后都查一遍，这是最省事的「体检」：

```

1 import mujoco
2 import numpy as np
3
4 WARNINGS = [
5     "mjWARN_BADQPOS", "mjWARN_BADQVEL", "mjWARN_BADQACC", "mjWARN_BADCTRL"
6     ,
7     "mjWARN_INERTIA", "mjWARN_CONTACTFULL", "mjWARN_CNSTRFULL", "
8     mjWARN_VGEOMFULL",
9 ]
10
11 def check(d):
12     ok = True
13     for name in WARNINGS:
14         try:
15             stat = d.warning[getattr(mujoco.mjtWarning, name)]
16         except AttributeError: # 版本不同，枚举名可能变化
17             continue
18         if stat.number:
19             ok = False
20             print(f"{name}: count={stat.number} lastinfo={stat.lastinfo}")

```

```

19     if not np.isfinite(d.qpos).all() or not np.isfinite(d.qvel).all():
20         ok = False
21         print("state contains NaN or Inf")
22     return ok

```

表 9.1: 常见警告的含义

警告	含义与处理
mjWARN_BADQPOS / BADQVEL / BADQACC / BADCTRL	状态量或控制量出现非有限值 (NaN/Inf), 即发散。先减小时间步长, 再检查质量与惯量是否合理、是否有初始穿插
mjWARN_INERTIA	惯量不满足物理约束。检查密度设置, 或对 URDF 使用 <code>balanceinertia</code>
mjWARN_CONTACTFULL	接触点数量超过上限 (<code>size</code> 里的 <code>nconmax</code>), 超出的接触被丢弃。减少不必要的碰撞对, 或调大上限
mjWARN_CNSTRFULL	约束行数超过上限, 同上处理
mjWARN_VGEOMFULL	可视化几何体数量超过 <code>mjvScene</code> 容量, 只影响显示

9.2.3 第三条：静态平衡检查

把机器人放在地面上、不加控制, 跑一小段时间。一个物理上合理的模型应当**基本静止**: 各关节速度保持在小量级, 位置漂移缓慢且单调。

```

1  import mujoco
2  import numpy as np
3
4  m = mujoco.MjModel.from_xml_path("scene.xml")
5  d = mujoco.MjData(m)
6  mujoco.mj_resetDataKeyframe(m, d, 0)
7
8  for _ in range(2000):
9      mujoco.mj_step(m, d)
10
11  print("max |qvel| =", np.abs(d.qvel).max())
12  print("drift      =", np.abs(d.qpos - d.qpos).max())  # 与初值比较请先另存副本
13  print("tool z     =", d.site("tool").xpos[2])

```

若速度持续增长, 说明模型里有互相穿透的碰撞体、或有无限位的自由度在重力下加速; 若一放手就「弹飞」, 多半是初始穿插导致巨大的排斥力 (官方文档在「发散」一节里正是把这种情况列为首要原因之一)。

9.3 稳定性与调参

9.3.1 参数速查

表 9.2: 常用的稳定性参数

参数	典型值	作用与调整方向
option/timestep	0.002 s	积分步长。发散时第一个要减小的参数；减半通常就能稳住，代价是仿真变慢
option/integrator	implicitfast	积分器。默认 Euler 最省；implicitfast 对阻尼项隐式处理，稳定性更好而开销增加不多；RK4 精度高但慢
joint/damping	0.1 ~ 1	关节黏性阻尼，抑制高频振荡的有效手段
joint/armature	0.01 ~ 0.1	关节等效惯量，改善数值条件；轻质连杆尤其需要
joint/frictionloss	视机构而定	干摩擦，用来抑制「缓慢滑移」
geom/friction	1 0.005 0.0001	滑动、扭转、滚动摩擦系数。打滑就调大第一项
geom/solref	0.02 1	约束的时间常数与阻尼比。值越大越「软」，接触越不容易抖
geom/solimp	0.9 0.95 0.001	约束阻抗。增大前两项可以明显减少缓慢滑移，但需要更小的时间步配合
option/impratio	1 ~ 10	法向与切向阻抗之比，配合 solimp 一起调，是减少滑移的主力

「滑移」不是摩擦不够 官方文档专门澄清了一个反直觉的现象：MuJoCo 的约束是软的，因此接触点出现缓慢滑移是模型本身的数学性质，而不是摩擦系数设小了。要抑制滑移，正确做法是增大 solimp 的阻抗、提高 impratio、换用 Newton 求解器，而不是盲目加大摩擦系数。而这类调整通常需要更小的时间步来保持稳定。

9.3.2 按症状查方向

表 9.3: 症状与调整方向

症状	处理顺序
直接发散（出现 NaN）	减小 <code>timestep</code> → 换 <code>implicitfast</code> → 检查质量与惯量 → 检查初始是否穿插
高频抖动	加 <code>damping / armature</code> ；放宽 <code>solref</code> ；确认碰撞网格没有重复
接触处缓慢滑动	提高 <code>solimp</code> 与 <code>impratio</code> ；必要时启用 <code>NoSlip</code> 后处理求解器（会牺牲凸性，谨慎）
相邻连杆互相卡住	碰撞体过大或凸包填实了凹处；改用更小的碰撞体或拆分
物体穿过地面	<code>timestep</code> 过大导致穿透；减小步长或加厚地面几何体
同样的初值每次结果不同	检查是否用了多线程/多进程而未固定随机种子；单次仿真本身是确定的

9.4 报错信息索引

表 9.4: 按报错信息定位问题

报错关键词	通常原因
<code>could not open file</code>	网格或纹理路径不对；见 8.1.2 节
<code>Error: repeated name</code>	同类元素重名
<code>Error: unknown element / unknown attribute</code>	XML 元素或属性拼写错误；URDF 里拼错的属性会被静默忽略
<code>mass and inertia of moving bodies must be larger than mjMINVAL</code>	某个移动物体的质量或惯量为 0（常见于只写了 <code>visual</code> 几何体的情况）
<code>inertia must be positive definite</code>	惯量非法；URDF 可用 <code>balanceinertia</code>
<code>mesh has ... faces / 翻转面</code>	网格法线方向不一致
<code>Error: ... qhull ...</code>	凸包计算失败，通常是网格退化或顶点共面严重；尝试减面
<code>Loading model error: ...</code>	加载阶段的通用前缀，后面的具体信息才是关键
<code>DLL load failed / 找不到 mujoco.dll</code>	运行环境问题，见 2.6 与 4.5 节

9.5 性能优化

1. **用 Release 构建：**Debug 版可能慢数倍。多配置生成器下别忘了 `--config Release`。
2. **减少碰撞对：**用 `contype/conaffinity` 关掉不可能接触的几何体之间的检测，用 `<contact><exclude>` 排除结构上永远不接触的物体对。这一步的收益通常最大。

3. **简化碰撞网格**：接触检测的开销与三角形数量相关；用基本体或减面网格做碰撞体。
4. **关掉不需要的传感器与可视化**：传感器每步都要计算，不需要的删掉。
5. **并行采样**：一个模型配多个 `mjData`，每个线程一个；Python 下可直接用 `mujoco.rollout`，把「循环 `mj_step`」放到 C++ 层并用线程池执行（注意 Python 绑定在调用 C 函数时会释放 GIL，但其余 Python 代码仍受 GIL 限制，因此多进程往往比多线程更划算）。
6. **上 GPU**：需要成千上万条并行轨迹时，改用 MJX（JAX 后端）或 MuJoCo Warp，它们消费同样的 `mjModel/mjData` 语义，可以无缝对照。
7. **换编译器**：官方指出 Windows 上 Clang 比 MSVC 更快；对性能敏感的项目值得一试（第六章）。

本指南到此结束 后续想深入，建议按这个顺序读官方文档：*Modeling*（建模语言与各类元素语义）→ *Computation*（求解器与数值细节）→ *XML Reference*（属性字典）→ *Programming*（C API 与可视化）。遇到具体问题，MuJoCo 的 GitHub Issues 里几乎都能搜到同样的报错。

附录 A 速查表与常见问题

A.1 工程目录模板

```
1 my_project/
2 |-- CMakeLists.txt      CMake 工程定义
3 |-- main.cc             C++ 入口
4 |-- scripts/
5 |   |-- run_sim.py      Python 仿真脚本
6 |   |-- check_model.py  质量、惯量、警告自检
7 |-- model/
8 |   |-- scene.xml       场景 (include 机器人 + 地面 + 光照 + 相机)
9 |   |-- robot.xml       机器人本体
10 |   |-- assets/        网格与纹理
11 |       |-- link1.STL
12 |       |-- link2.STL
13 |-- build/             构建目录 (不提交到版本库)
```

三条约定值得坚持：模型与代码分离、场景与本体分离、构建目录不进版本库（加 .gitignore）。

A.2 环境变量与路径速查

表 1.1: 常用环境变量与路径

名称	用途
PATH	加入 <SDK>\bin 后，程序才能找到 mujoco.dll
mujoco_DIR	CMake 变量，指向含 mujocoConfig.cmake 的目录
CMAKE_PREFIX_PATH	CMake 变量，指向安装前缀：比 mujoco_DIR 更宽松
MUJOCO_SDK	自定义变量（建议），在 CMakePresets.json 里用 \$env{...} 引用
MUJOCO_PATH / MUJOCO_PLUGIN_PATH	从源码构建 Python 绑定时使用，普通用户不需要

A.3 命令速查

```
1  :: Python
2  python -m venv .venv
3  .\.venv\Scripts\Activate.ps1
4  pip install mujoco
5  python -c "import mujoco; print(mujoco.__version__)"
6  python -m mujoco.viewer --mjcf=scene.xml
7
8  :: 预编译库
9  .\bin\simulate.exe ..\model\humanoid\humanoid.xml
10
11 :: CMake (多配置生成器)
12 cmake -S . -B build -G "Visual Studio 17 2022" -A x64 -DCMAKE_PREFIX_PATH=
    D:/mujoco/install
13 cmake --build build --config Release
14 .\build\Release\demo.exe
15
16 :: CMake (Ninja, 需在 VS 开发者命令行中)
17 cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -
    DCMAKE_PREFIX_PATH=D:/mujoco/install
18 cmake --build build
19
20 :: CMake Presets
21 cmake --preset vs2022-release
22 cmake --build --preset vs2022-release
23
24 :: 源码安装
25 git clone https://github.com/google-deepmind/mujoco.git
26 cmake -S mujoco -B mujoco/build -G "Visual Studio 17 2022" -A x64 -
    DCMAKE_INSTALL_PREFIX=D:/mujoco/install
27 cmake --build mujoco/build --config Release
28 cmake --install mujoco/build --config Release
29
30 :: 网格工具
31 pip install --upgrade obj2mjcf
32 obj2mjcf --obj-dir assets --save-mjcf --compile-model
```

A.4 版本与默认值速查

表 1.2: 版本基准与工具链要求

项目	数值
本指南版本基准	MuJoCo 3.14.0 (2026-09-22 发布)
Windows 安装包	mujoco-3.14.0-windows-x86_64.zip (约 22 MB)
Python	官方 wheel 覆盖 3.10 ~ 3.15, 仅 64 位; 实践推荐 3.12 / 3.13
C++ 编译器	Visual Studio 2019 或更高; C++17; 需支持 AVX 的 CPU
Windows SDK	10.0.22000 及以上
CMake	官方源码下限 3.16, 本指南建议 3.24 及以上
CMake 导入目标	mujoco::mujoco; 没有 mujoco::glfw (GLFW 是 glfw)
OpenGL	1.5 兼容模式 + ARB_framebuffer_object、 ARB_vertex_buffer_object; Windows 上由系统与显卡驱动提供

表 1.3: 必须记住的默认值与约定

项目	默认值 / 约定
单位	引擎 不强制 单位制, 推荐米-千克-秒 (MKS)
重力	0 0 -9.81
默认密度	1000 (水的密度), 用于由几何体反推质量
时间步长	0.002 s
世界朝向	z 轴向上, 右手系
四元数顺序	$wxyz$, 单位四元数 "1 0 0 0"
角度	MJCF 默认度; URDF 一律弧度; mjModel/mjData 内一律弧度
坐标	所有元素的位置与姿态都相对父物体表达
接触	只使用几何体的 凸包 ; 父子物体之间默认不检测碰撞
颜色	网格本身无色, 颜色来自引用它的 geom 的材质

A.5 MJCF 元素速查

表 1.4: 最常用的顶层元素

元素	作用
compiler	编译期选项: <code>angle</code> (度/弧度)、 <code>meshdir/assetdir</code> 、 <code>discardvisual</code> 等
option	运行期选项: <code>timestep</code> 、 <code>gravity</code> 、 <code>integrator</code> 、 <code>impratio</code>
size	各种上限: <code>nconmax</code> 、 <code>nkey</code> 等
default	默认值与默认类, 作用类似 CSS
asset	资源: <code>mesh</code> 、 <code>texture</code> 、 <code>material</code> 、 <code>hfield</code> 、 <code>skin</code>
worldbody	运动学树的根, 含 <code>body/geom/site/light/camera</code>
actuator	执行器: <code>motor</code> 、 <code>position</code> 、 <code>velocity</code> 、 <code>general</code> 等
sensor	传感器: <code>jointpos</code> 、 <code>jointvel</code> 、 <code>imu</code> 、 <code>force</code> 、 <code>rangefinder</code> 等
tendon	肌腱 (定长与空间路径), 可做耦合与限位
equality	等式约束: <code>connect</code> 、 <code>weld</code> 、 <code>joint</code> 、 <code>tendon</code>
contact	接触微调: <code>pair</code> (指定接触对) 与 <code>exclude</code> (排除接触)
keyframe	关键帧: 保存完整的初始状态

A.6 资源

表 1.5: 官方与社区资源

资源	地址
官网	https://mujoco.org
源码仓库	https://github.com/google-deepmind/mujoco
发布下载	https://github.com/google-deepmind/mujoco/releases
文档	https://mujoco.readthedocs.io
XML 属性字典	https://mujoco.readthedocs.io/en/stable/XMLreference.html
Python 教程笔记本	https://github.com/google-deepmind/mujoco/blob/main/python/tutorial.ipynb
模型库 Menagerie	https://github.com/google-deepmind/mujoco_menagerie
SolidWorks 导出插件	https://github.com/ros/solidworks_urdf_exporter
插件使用教程	https://wiki.ros.org/sw_urdf_exporter
OBJ 处理工具	https://github.com/kevinzakka/obj2mjcf
MeshLab (减面/修面)	https://www.meshlab.net

A.7 常见问题

1. 到底该学 Python 还是 C++?

先用 Python 把模型和控制逻辑跑通, 再按需要把性能敏感的部分用 C++ 重写。

两者的模型文件与数值结果是一致的。

2. 必须装 Visual Studio 吗？

只有走 C++ 路线才需要。Python 路线下 `pip install mujoco` 自带引擎，连编译器都不需要。

3. 没有独立显卡能跑吗？

纯计算可以，而且性能只取决于 CPU。只有需要窗口显示或离屏渲染时才需要可用的 OpenGL 驱动。

4. Windows 上能用 EGL / OSMesa 做无头渲染吗？

不能。这两个后端只在 Linux 上提供；Windows 与 macOS 使用系统自带的 OpenGL 实现，因此必须有一个（可以是隐藏的）窗口上下文。

5. 能不能直接加载 SolidWorks 文件？

不能。必须先从 SolidWorks 导出为 OK 的中间格式（URDF 或 OBJ/STL），再由 MuJoCo 编译（第七章）。

6. 一个 MJCF 里可以放多个模型吗？

一个 MJCF 就是一个模型。要同时仿真两个机器人，把它们写进同一个模型的 `worldbody` 下即可；要分别仿真，则用多个 `MjModel`。

7. 怎么把仿真录成视频？

用 `mujoco.Renderer` 逐帧取图，再用 `imageio` 写成 MP4；或者用预编译包里的 `record` 示例程序（C++）。

8. 为什么仿真结果和 Gazebo / Isaac Sim 不一致？

三个引擎的接触模型、积分器与默认参数都不同。MuJoCo 的约束是软的，接触处会有可观测的柔顺与滑移；这在 MuJoCo 内部是自洽的，拿来作控制与学习都没有问题，但不要指望它与其他引擎逐位对齐。

9. 要不要装 mujoco-py？

不要。它是为旧版引擎写的第三方封装，已废弃；新版 `mujoco` 包不需要任何许可证密钥。

10. 模型建好之后，最该做的一件事是什么？

在程序里查一遍 `data.warning` 与 `body_mass`（第九章）。这两项检查能在几分钟内发现绝大多数建模错误。